Code-Space Response Oracles: Generating Interpretable Multi-Agent Policies with Large Language Models

Daniel Hennes[*1], **Zun Li**[*1], **John Schultz**[1] **and Marc Lanctot**[1]
[*]Equal contributions, [1]Google DeepMind

Recent advances in multi-agent reinforcement learning, particularly Policy-Space Response Oracles (PSRO), have enabled the computation of approximate game-theoretic equilibria in increasingly complex domains. However, these methods rely on deep reinforcement learning oracles that produce 'black-box' neural network policies, making them difficult to interpret, trust or debug. We introduce Code-Space Response Oracles (CSRO), a novel framework that addresses this challenge by replacing RL oracles with Large Language Models (LLMs). CSRO reframes the best response computation as a code generation task, prompting an LLM to generate policies directly as human-readable code. This approach not only yields inherently interpretable policies but also leverages the LLM's pretrained knowledge to discover complex, human-like strategies. We explore multiple ways to construct and enhance an LLM-based oracle: zero-shot prompting, iterative refinement and *AlphaEvolve*, a distributed LLM-based evolutionary system. We demonstrate that CSRO achieves performance competitive with baselines while producing a diverse set of explainable policies. Our work presents a new perspective on multi-agent learning, shifting the focus from optimizing opaque policy parameters to synthesizing interpretable algorithmic behavior.

arXiv:2603.10098v1 [cs.GT] 10 Mar 2026

1. Introduction

The strategic interaction of multiple autonomous agents is a cornerstone of artificial intelligence, with applications ranging from economic modeling and autonomous driving to cybersecurity and recreational games (Shoham and Leyton-Brown, 2008). A central goal in this field is to develop algorithms that compute robust and effective strategies. The predominant theoretical framework for analyzing such systems is game theory. However, computing exact equilibria in large, complex games is often intractable, motivating the development of iterative methods to find approximate solutions.

Among the most successful of these methods are Policy-Space Response Oracles (PSRO) (Bighashdel et al., 2024; Lanctot et al., 2017). PSRO and its variants have achieved state-of-the-art performance in notoriously difficult games, such as Barrage Stratego (Mcaleer et al., 2020) and StarCraft (Vinyals et al., 2019). PSRO iteratively builds a set of policies by computing a best response to the current meta-strategy. Standard implementations rely on deep reinforcement learning (RL) oracles, which train a neural network to approximate a best response through extensive interaction with a population of opponents. While powerful, this approach results in policies that are opaque, "black-box" models. This lack of interpretability prevents strategy verification, and forms a significant barrier to deploying such agents in high-stakes, real-world applications where explainability is crucial. Furthermore, training these RL oracles is often sample-inefficient, requiring millions or billions of game simulations to converge.

To address the trade-off between performance and interpretability, we propose **Code-Space Response Oracles (CSRO)**. We reframe best-response computation as a program synthesis task, replacing the Deep RL oracle with a Large Language Model (LLM) that generates policies as executable source code. By providing the LLM with the game rules, an API for interacting with the environment,

Corresponding author(s): hennes@google.com, lizun@google.com
© 2026 Google DeepMind. All rights reserved

代码空间响应预言机：利用大语言模型生成可解释多智能体策略

Daniel Hennes[*1]、**Zun Li**[*1]、**John Schultz**[1] **和 Marc Lanctot**[1]
[*]同等贡献， [1]Google DeepMind

多智能体强化学习的最新进展，尤其是策略空间响应预言机（PSRO）的发展，使得在日益复杂的领域中计算近似博弈论均衡成为可能。然而，这些方法依赖于深度强化学习预言机，其生成的"黑箱"神经网络策略难以解释、信任或调试。我们提出了代码空间响应预言机（CSRO），这一新框架通过用大语言模型替代强化学习预言机来应对这一挑战。CSRO 将最优响应计算重新表述为代码生成任务，通过提示词引导大语言模型直接以人类可读代码的形式生成策略。这种方法不仅能产生本身就具有可解释性的策略，还能利用大语言模型的预训练知识来发现复杂的类人策略。我们探索了构建和增强基于大语言模型的预言机的多种方式：零样本提示、迭代改进以及*AlphaEvolve*——一种分布式的基于大语言模型的演化系统。我们证明，CSRO 在生成多样化可解释策略的同时，能够达到与基线方法相媲美的性能。我们的工作为多智能体学习提供了新的视角，将重点从优化不透明的策略参数转向合成可解释的算法行为。

arXiv:2603.10098v1 [cs.GT] 10 Mar 2026

1. 引言

多个自主智能体之间的策略交互是人工智能的基石，其应用涵盖经济建模、自动驾驶、网络安全和休闲游戏等领域（Shoham和Leyton-Brown，2008年）。该领域的核心目标之一是开发能够计算稳健且有效策略的算法。分析此类系统的主流理论框架是博弈论。然而，在大规模复杂博弈中计算精确均衡往往在计算上不可行，因此推动了用于寻找近似解的迭代方法的发展。

在这些方法中，策略空间响应预言机（策略空间响应预言机）是最成功的方法之一（Bighashdel等人，2024年；Lanctot等人，2017年）。策略空间响应预言机及其变体在一些公认极具挑战性的博弈中取得了最先进的性能，例如Barrage Stratego（Mcaleer等人，2020年）和星际争霸（Vinyals等人，2019年）。策略空间响应预言机通过计算当前元策略的最优响应，迭代构建策略集合。标准实现依赖深度强化学习（强化学习）预言机，通过与对手种群进行大量交互来训练神经网络，以逼近最优响应。尽管这种方法能力强大，但其生成的策略是不透明的"黑箱"模型。这种可解释性的缺失使策略验证无法进行，也成为将此类智能体部署到高风险现实应用中的重大障碍，因为这些应用对可解释性有着至关重要的要求。此外，这些强化学习预言机的训练往往样本效率低下，需要数百万甚至数十亿次博弈仿真才能收敛。

为解决性能与可解释性之间的权衡问题，我们提出了**代码空间响应预言机（CSRO）**。我们将最优响应计算重新表述为程序合成任务，用大语言模型（LLM）取代深度强化学习预言机，以可执行源代码的形式生成策略。通过向大语言模型提供博弈规则、用于与环境交互的应用程序编程接口，

通讯作者：*hennes@google.com, lizun@google.com*
© 2026年 Google DeepMind。保留所有权利

and descriptions of opponent strategies, we prompt it to write a programmatic best response. CSRO leverages the vast knowledge of logic, planning, and strategy embedded within pre-trained LLMs, enabling them to generate sophisticated, human-like policies, dramatically reducing the need for costly exploration.

CSRO generalizes a recent demonstration called LLM-PSRO (Bachrach et al., 2025). LLM-PSRO relies on open-loop "best-of-N" sampling using full opponent source code, CSRO introduces two critical extensions for scalability and robustness. First, we replace passive sampling with an *iterative refinement loop* (e.g., via evolutionary algorithms) that actively optimizes strategies based on meta-game performance. Second, we address context limits via *context abstraction*; rather than ingesting raw code from all opponents, CSRO summarizes strategies in natural language and filters inputs. This allows our method to scale to complex games where full source code would exceed context windows.

This paper makes several contributions, primarily introducing CSRO as a novel framework utilizing LLMs as code-generating oracles to compute approximate equilibria. We demonstrate that while LLMs can generate strategies zero-shot, performance is significantly enhanced by integrating evolutionary refinement method, such as AlphaEvolve (Novikov et al., 2025; Romera-Paredes et al., 2024), to iteratively optimize code based on closed-loop feedback. This approach yields policies that are fully interpretable, represented by commented source code. Finally, unlike prior work limited to internal comparisons (Bachrach et al., 2025), we rigorously validate CSRO by benchmarking against standardized external populations and game-theoretic solvers, showing that code-generation oracles can compete with mature baselines in established domains.

2. Code Space Response Oracles

In this section, we first provide a brief overview of the standard PSRO algorithm, which forms the foundation of our work. We then formally introduce our proposed method, CSRO, detailing how an LLM is employed as a code-generating oracle to produce interpretable, programmatic policies.

2.1. Preliminaries

PSRO is an iterative algorithm for computing approximate Nash equilibria in multi-agent games. In this paper we focus on two-player symmetric zero-sum games, but the methodology directly transfers to general game domains. Consider a two-player symmetric game where each player $i \in \{0, 1\}$ shares a common set of strategies Π with the same utility function u. A strategy, or a policy, $\pi \in \Pi$, is a mapping from observations to a probability distribution over actions. A player playing π receives $u(\pi, \pi')$ against an opponent playing π'; the opponent receives $u(\pi', \pi) = -u(\pi, \pi')$ according to the zero-sum condition. The PSRO algorithm in a symmetric game setting maintains a single set of policies P. At each iteration k, the algorithm first solves for a symmetric equilibrium mixture σ over the current policies in an empirical meta-game. Then, from the viewpoint of any player (since they are symmetric), a best response oracle computes a policy π^* that maximizes expected utility against the opponent meta-strategy σ:

$$\pi^* \in \arg\max_{\pi} \mathbb{E}_{\pi' \sim \sigma}[u(\pi, \pi')] \tag{1}$$

This new policy is added to the policy set P, and the process repeats. In standard PSRO, the oracle is a deep reinforcement learning algorithm that produces an opaque neural network policy.

以及对手策略的描述，我们通过提示词引导其编写程序化最优响应。CSRO 利用预训练大语言模型中蕴含的丰富逻辑、规划和策略知识，使其能够生成复杂且类人的策略，从而大幅减少对高成本探索的需求。

CSRO 推广了近期一项名为大语言模型策略空间响应预言机的验证性研究（Bachrach 等人，2025年）。大语言模型策略空间响应预言机依赖使用完整对手源代码的开环“N中选优”采样，而 CSRO 引入了两项关键扩展，以提升可扩展性和鲁棒性。首先，我们用迭代优化循环（例如通过进化算法实现）取代被动采样，根据元博弈表现主动优化策略。其次，我们通过上下文抽象应对上下文限制；CSRO 不会直接摄入所有对手的原始代码，而是用自然语言总结策略，并对输入进行筛选。这使我们的方法能够扩展到完整源代码会超出上下文窗口的复杂博弈。

本文做出了多项贡献，其中最主要的是提出了 CSRO 这一新颖框架，利用大语言模型作为代码生成预言机来计算近似均衡。我们表明，虽然大语言模型能够以零样本方式生成策略，但通过集成 AlphaEvolve 等进化优化方法（Novikov 等人，2025年；Romera-Paredes 等人，2024年），基于闭环反馈迭代优化代码，可以显著提升性能。这种方法生成的策略以带注释的源代码表示，具有完全的可解释性。最后，不同于以往仅限于内部比较的工作（Bachrach 等人，2025年），我们通过与标准化外部种群和博弈论求解器进行基准比较，对 CSRO 进行了严格验证，结果表明，代码生成预言机能够在成熟领域中与成熟的基线方法相媲美。

2. 代码空间响应预言机

本节首先简要概述标准PSRO算法，该算法是我们工作的基础。随后，我们正式介绍所提出的方法CSRO，详细说明如何将大语言模型用作代码生成预言机，以生成可解释的程序化策略。

2.1. 预备知识

策略空间响应预言机是一种用于计算多智能体博弈中近似纳什均衡的迭代算法。本文重点研究双人对称零和博弈，但该方法可以直接推广至一般博弈领域。考虑一个双人对称博弈，其中每位玩家 $i \in \{0,1\}$ 共享同一个策略集 Π，并具有相同的效用函数 u。策略 $\pi \in \Pi$ 是从观测到动作概率分布的映射。采用 π 的玩家在面对采用 π' 的对手时，获得的效用为$u(\pi, \pi')$；根据零和条件，对手获得的效用为 $u(\pi', \pi) = -u(\pi, \pi')$。对称博弈设定下的PSRO算法维护单一策略集 P。在每次迭代 k 中，算法首先在经验元博弈中求解当前策略上的对称均衡混合策略 σ。然后，从任一玩家的视角出发（因为双方是对称的），最佳响应预言机计算一个策略 π^*，使其针对对手元策略 σ的期望效用最大化：

$$\pi^* \in \arg\max_{\pi} \mathbb{E}_{\pi' \sim \sigma}[u(\pi, \pi')] \tag{1}$$

这一新策略被加入策略集 P，随后重复这一过程。在标准策略空间响应预言机中，预言机是一种深度强化学习算法，它生成不透明的神经网络策略。

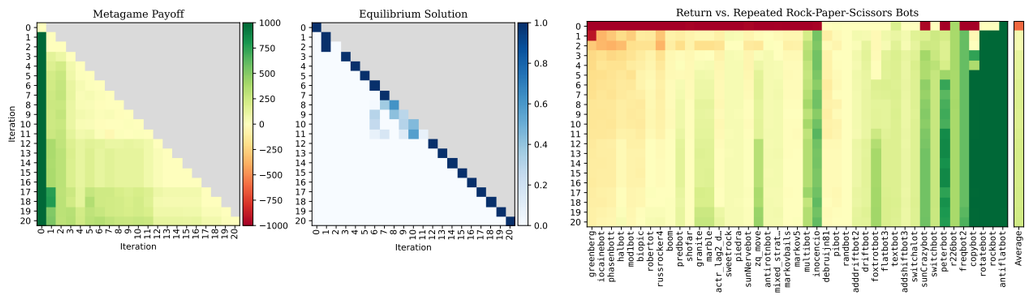

Figure 1 | Example run of CSRO in Repeated Rock-Paper-Scissors.

2.2. Code Policies

The central innovation of CSRO is the replacement of the deep RL oracle with an LLM that synthesizes policies as executable code. This reframes the best response computation from a process of numerical optimization to one of programmatic reasoning and generation. In our framework, a policy π is no longer an opaque neural network but a (stateful) code policy, such as a Python function, that maps observations to actions.

The primary mechanism for guiding the LLM is a carefully constructed prompt, which is dynamically generated at each iteration of the algorithm. This prompt provides the LLM with all necessary context to formulate a strategy. It begins with a natural language description of the game's rules and objectives, alongside a precise API specification for the policy function to ensure the generated code is executable. A crucial component is the description of the opponent's current meta-strategy, σ. Since opponent policies in CSRO are also programmatic, their source code can be directly included in the prompt. To manage prompt complexity when facing a large mixture of policies, we can in addition to — or instead of the code — provide a high-level summary of the opponents' collective behavior, a summary that can itself be generated by prompting the LLM to analyze the set of code policies. The prompt concludes with a clear directive to generate a best response and can optionally include an instruction to produce self-explaining code with detailed comments and a docstring describing the intended strategy.

2.3. CSRO Algorithm

The complete CSRO algorithm is detailed in Algorithm 1. The overall structure follows the iterative loop of PSRO, but the oracle step is replaced by an LLM-based code generation process. At each iteration, a new programmatic policy is generated, and added to the policy set, enriching the meta-game with increasingly sophisticated strategies.

2.4. Oracle Refinement Mechanisms

The CSRO framework supports two distinct, complementary mechanisms for improving the quality of generated policies using in-context learning, as detailed in Algorithm 1. These mechanisms operate on different timescales to provide both immediate tactical corrections and long-term strategic guidance.

2.4.1. Cross-Iteration Strategic Adaptation

At the beginning of each iteration k, the oracle is provided with deep strategic context by analyzing the programmatic policies that constitute the current meta-game equilibrium σ. The `construct_prompt`

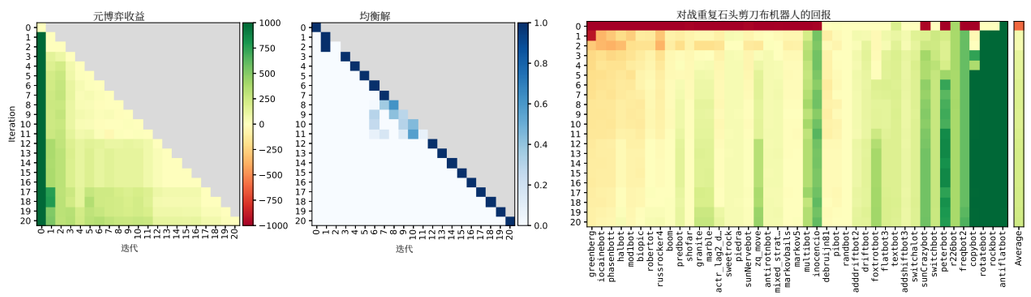

图 1 | CSRO在重复石头剪刀布博弈中的运行示例。

2.2. 代码策略

CSRO 的核心创新在于，用能够将策略合成为可执行代码的大语言模型替代深度强化学习预言机。这将最优响应计算从数值优化过程重构为程序化推理与生成过程。在我们的框架中，策略 π 不再是不透明的神经网络，而是将观测映射为动作的（有状态）代码策略，例如一个 Python函数。

引导大语言模型的主要机制是精心构建的提示词，该提示词在算法的每次迭代中动态生成。此提示词为大语言模型提供制定策略所需的全部上下文。它首先以自然语言描述博弈规则和目标，同时提供策略函数的精确API规范，以确保生成的代码可执行。其中一个关键组成部分是对对手当前元策略 σ的描述。由于CSRO中的对手策略也以程序形式表示，因此可以将其源代码直接纳入提示词。面对大量策略的混合时，为控制提示词的复杂度，除了提供代码之外，我们还可以提供对手整体行为的高层次概述，或用该概述替代代码；概述本身也可以通过提示大语言模型分析这组代码策略来生成。提示词最后会明确要求生成最优响应，还可以选择加入一条指令，要求生成自解释代码，并附上详细注释和描述预期策略的文档字符串。

2.3. CSRO算法

完整的CSRO算法详见算法1。其整体结构遵循策略空间响应预言机的迭代循环，但将预言机步骤替换为基于大语言模型的代码生成过程。在每次迭代中，都会生成一个新的程序化策略，并将其加入策略集，以日益复杂的策略丰富元博弈。

2.4. 预言机改进机制

CSRO框架支持两种不同且互补的机制，通过上下文学习提高所生成策略的质量，具体见算法1。这些机制在不同时间尺度上发挥作用，既提供即时的战术修正，也提供长期的战略指导。

2.4.1. 跨迭代战略适应

在每次迭代 k 开始时，通过分析构成当前元博弈均衡 σ的程序化策略，为预言机提供深入的战略上下文。构建_提示词

Algorithm 1: Code-Space Response Oracles.

Input: G: symmetric zero-sum game, K: max. iterations, M: max. refinement budget

1 $P \leftarrow \{\pi_{\text{initial}}\}$
2 **for** $k \leftarrow 1$ **to** K **do**
3 $U \leftarrow$ `compute_payoff_matrix`(P)
4 $\sigma \leftarrow$ `compute_meta_equilibrium`(U)
5 $prompt \leftarrow$ `construct_prompt`(G, σ, P)
6 $\pi' \leftarrow$ `llm_oracle`$(prompt)$
7 $u \leftarrow$ `evaluate_policy`(π', σ, P)
8 $j \leftarrow 0$
9 **while** ***not*** `terminated`*(u, j, M)* **do**
10 $j \leftarrow j + 1$
11 $prompt \leftarrow$ `update_prompt`$(prompt, \pi', u)$
12 $\pi' \leftarrow$ `llm_oracle`$(prompt)$
13 $u \leftarrow$ `evaluate_policy`(π', σ, P)
14 **end**
15 $P \leftarrow P \cup \{\pi'\}$
16 **end**
17 **return** P, σ

function inspects the source code of the opponent policies currently active in the meta-game (i.e., those with a non-zero probability in σ). The *source code* of each opponent can be directly included in the prompt or alternatively summarized by another LLM call to provide a high-level *description* of the strategic behavior the opponent code implements. To keep the context length manageable we can, e.g., filter by a threshold on *minimum support* or only include the *top-k* opponents according to the meta-game equilibrium. The provided opponents' source code or descriptions guide the generation of the next candidate policy.

2.4.2. Intra-Iteration Policy Refinement

The second mechanism is an inner feedback loop designed to ensure that any new policy π' is a robust best response to the current meta-game σ. We here consider three variants that differ in how the refinement stage in lines 9-14 are implemented:

1. *ZeroShot*: We prompt the LLM to directly generate a program in zero-shot fashion. This corresponds to the case where the refinement stage (Algorithm 1: line 9–14) is never entered.
2. *LinearRefinement*: We start with an initial candidate policy whose expected utility u against σ is immediately evaluated. If the policy is found to be losing ($u < 0$), a conditional refinement loop is triggered. The LLM is then tasked with regenerating the policy based on this feedback. The program is updated if the score is increased. This cycle of evaluation and refinement continues until the policy achieves a non-negative utility or a maximum refinement budget M is reached. We call it "linear refinement" because all the computations are exercised in a single thread.
3. *AlphaEvolve*: AlphaEvolve (Novikov et al., 2025; Romera-Paredes et al., 2024) is a large-scale distributed system which uses an LLM to mutate programs in multiple threads and uses a score function to guide an evolutionary search procedure. To ensure diversity, programs are clustered into different subpopulations which are evolved independently. Each evolution thread continuously samples past programs and prompts the LLM to generate new modifications to them in favor of high-scored variants. AlphaEvolve has been shown effective in finding novel and better algorithms in

算法 1：代码空间响应预言机。

输入： G：对称零和博弈， K：最大迭代次数， M：最大优化预算

1 $P \leftarrow \{\pi_{初始}\}$
2 **对于** $k \leftarrow 1$ **到** K **执行**
3 $U \leftarrow$计算_收益_矩阵(P)
4 $\sigma \leftarrow$计算_元_均衡(U)
5 $prompt \leftarrow$ 构建_提示词(G, σ, P)
6 $\pi' \leftarrow$ 大语言模型_预言机$(prompt)$
7 $u \leftarrow$ 评估_策略(π', σ, P)
8 $j \leftarrow 0$
9 **当** 未 终止*(u, j, M)* **时执行**
10 $j \leftarrow j + 1$
11 $prompt \leftarrow$ 更新_提示词$(prompt, \pi', u)$
12 $\pi' \leftarrow$ 大语言模型_预言机$(prompt)$
13 $u \leftarrow$ 评估_策略(π', σ, P)
14 **end**
15 $P \leftarrow P \cup \{\pi'\}$
16 **结束**
17 **返回** P $, \sigma$

函数检查当前在元博弈中活跃的对手策略（即在 σ中具有非零概率的策略）的源代码。每个对手的源代码 可以直接包含在提示词中，也可以通过另一次大语言模型调用进行总结，以提供对手代码所实现的策略行为的高层次描述 。为使上下文长度保持在可控范围内，例如，我们可以按最小支持度 阈值进行筛选，或仅包含根据元博弈均衡选出的前 k 个 对手。所提供的对手源代码或描述将指导下一个候选策略的生成。

2.4.2. 迭代内策略细化

第二种机制是一个内部反馈循环，旨在确保任何新策略 π' 都是针对当前元博弈 σ 的稳健最优回应。我们在此考虑三种变体，它们的区别在于第 9–14 行中优化阶段的实现方式：

1. 零样本：我们通过提示词让大语言模型以零样本方式直接生成程序。这对应于从不进入优化阶段（算法 1：第 9–14 行）的情况。
2. *LinearRefinement*：我们从一个初始候选策略开始，并立即评估其对抗 σ 时的期望效用 u 。如果发现该策略处于劣势（$u < 0$），则触发条件优化循环。随后，大语言模型根据这一反馈重新生成策略。如果得分提高，则更新程序。这一评估与优化循环持续进行，直到策略达到非负效用，或达到最大优化预算 M 。我们将其称为“线性优化”，因为所有计算都在单线程中执行。

3. *AlphaEvolve*：AlphaEvolve（Novikov 等人，2025年；Romera-Paredes 等人，2024年）是一个大规模分布式系统，利用大语言模型在多线程中对程序进行变异，并利用评分函数指导进化搜索过程。为确保多样性，程序被聚类到不同的子种群中，各子种群独立演化。每个进化线程持续对历史程序进行采样，并通过提示词引导大语言模型对这些程序生成新的修改，以倾向于获得高分变体。已有研究表明，AlphaEvolve 能够有效发现新颖且更优的算法，其应用领域包括

math, optimization (Novikov et al., 2025) and cognitive science domains (Castro et al., 2025). We consider AlphaEvolve perfectly suitable for our purpose of computing best response programs using an estimate of $\mathbb{E}_{\pi' \sim \sigma}[u(\pi, \pi')]$ as the score function for the evolutionary search.

3. Experiments

To validate the effectiveness and interpretability of CSRO, we conduct a series of experiments designed to answer the following research questions: (1) Does CSRO converge to a low-exploitability equilibrium? (2) Can the LLM oracle generate effective, non-trivial strategies in a zero-shot or few-shot manner? (3) Are the programmatic policies generated by CSRO demonstrably more interpretable than the neural network policies produced by traditional oracles?

3.1. Environments

We evaluate our framework on two standard analytical games, chosen to test different aspects of strategic reasoning. Both environment implementations are based on OpenSpiel (Lanctot et al., 2019).

Repeated Rock-Paper-Scissors (RRPS): In this repeated version, players repeat the standard stage game for 1000 consecutive rounds (*throws*). Although playing according to a uniform random policy at each round is known to be a Nash equilibrium and is unexploitable, it fails to exploit patterns in the moves of suboptimal opponents. Therefore, RRPS has been a very informative benchmark for multiagent learning and opponent modeling (Billings, 2000a,b; Knoll et al.; Lanctot et al., 2023).

Repeated Leduc hold'em poker: In this environment, players play Leduc hold'em for 100 rounds (*hands*); the dealer role alternates between consecutive rounds, with the first-round-dealer randomly decided between the two players. Leduc hold'em is a popular benchmark for research in imperfect information games, introduced in (Southey et al., 2005). It is a two-player, zero-sum game with a small six-card deck (two suits of Jack, Queen, and King). The game features two betting rounds, with a single public community card revealed at the flop. Its structure, which combines private and public information across multiple decision points, requires more sophisticated stateful reasoning than simpler variants like Kuhn poker, making it an excellent environment to test the LLM's ability to generate complex, multi-step strategies involving concepts like bluffing and value betting. The repeated version of Leduc hold'em is a more realistic setting than the one-shot version; and it further adds another strategic layer where an agent is more effective if it identifies strategic patterns of the opponent and is thus able to exploit them (similar to RRPS).

3.2. Evaluation Population

In RRPS, we evaluate CSRO code-policies against 43 hand-coded, *heuristic strategies* from the international RRPS competitions (Billings, 2000a,b). These bots range from simple, stateless strategies like `randbot` (uniform random) and `rockbot` (always rock) to bots playing fixed sequences like `pibot`. Other approaches use limited memory, such as `copybot` (beating the opponent's last move) or `driftbot` (biasing actions based on recent play). More sophisticated entrants rely on historical observations, using methods like frequency analysis (`freqbot2`), Markov chain predictions (`markov5`), or complex rule-based systems (`inocencio`). Some bots switch between different internal policies based on profitability (`multibot`), while innovative designs include a neural network-like system (`sunNervebot`) and a cognitive architecture (`actr_lag2_decay`). The competition winners, `iocainebot` and `greenberg`, succeeded by maintaining a large set of different predictors and counter-strategies, selecting the one that proved most effective as the match progressed. This

数学、优化（Novikov 等人，2025年）和认知科学领域（Castro 等人，2025年）。我们认为，AlphaEvolve 完全适合我们的目标，即使用 $\mathbb{E}_{\pi' \sim \sigma}[u(\pi, \pi')]$ 的估计值作为进化搜索的评分函数，以计算最佳响应程序。

3. 实验

为验证 CSRO 的有效性和可解释性，我们开展了一系列实验，旨在回答以下研究问题：(1) CSRO 是否能收敛到可利用度较低的均衡？(2) 大语言模型预言机能否以零样本或少样本方式生成有效且非平凡的策略？(3) CSRO 生成的程序化策略是否明显比传统预言机生成的神经网络策略更具可解释性？

3.1. 环境

我们在两种标准解析博弈上评估所提出的框架，选择这两种博弈是为了测试策略推理的不同方面。两种环境的实现均基于 OpenSpiel（Lanctot等人，2019年）。

重复石头剪刀布博弈（RRPS）：在这一重复博弈版本中，玩家连续进行标准阶段博弈，共计 1000 个回合（出招）。尽管已知每回合采用均匀随机策略构成纳什均衡且无法被利用，但该策略无法利用次优对手动作中的规律。因此，RRPS 一直是多智能体学习和对手建模中极具参考价值的基准（Billings，2000年a，b；Knoll 等人；Lanctot等人，2023年）。

重复勒杜克德州扑克：在此环境中，玩家进行 100 回合的 Leduc 扑克（手牌）；庄家角色在相邻回合间交替，首回合的庄家由两名玩家随机决定。Leduc 扑克是不完美信息博弈研究中广泛使用的基准，最早由（Southey 等人，2005年）提出。这是一种双人零和博弈，使用一个小型六张牌组（杰克、王后和国王各有两种花色）。该博弈设有两个下注轮，并在翻牌时揭示一张公共牌。其结构在多个决策点上结合私有信息与公共信息，相比库恩扑克等更简单的变体，需要更复杂的有状态推理，因此非常适合用于测试大语言模型生成复杂多步骤策略的能力，这些策略涉及诈唬和价值下注等概念。与单次版本相比，重复勒杜克扑克的设定更贴近现实；它还增加了一个策略层面：如果智能体能够识别对手的策略模式并加以利用，就能取得更好的表现（类似于 RRPS）。

3.2. 评估种群

在RRPS中，我们将CSRO代码策略与来自国际RRPS竞赛的43种手工编写的启发式策略进行对比评估（Billings, 2000a,b）。这些机器人涵盖了从randbot（均匀随机）和rockbot（始终出石头）等简单的无状态策略，到pibot等采用固定序列的机器人。其他方法使用有限记忆，例如copybot（选择能击败对手上一步出招的动作）或driftbot（根据近期对局调整动作偏好）。更复杂的参赛机器人依赖历史观测，采用频率分析（freqbot2）、马尔可夫链预测（markov5）或复杂的基于规则的系统（inocencio）等方法。一些机器人根据收益情况在不同的内部策略之间切换（multibot），而创新设计则包括类神经网络系统（sunNervebot）和认知架构（actr_lag2_decay）。竞赛获胜者iocainebot和greenberg通过维护大量不同的预测器和反制策略，并随着比赛进程选择被证明最有效的策略而取得成功。这一

population of bots represent a diverse set of RRPS strategies which is useful in terms of measuring the generalization and exploitability of agents (Lanctot et al., 2023). An example run of CSRO in the RRPS environment is visualized in Figure 1, showing the evolution of payoffs and the equilibrium solution over iterations as well as returns against the bot population.

For repeated Leduc Poker, we compute a Nash equilibrium for the single-hand game using *Counterfactual Regret Minimization* + (CFR+) (Tammelin, 2014). We run CFR+ for 10 000 iterations and use the weighted averaged strategy. We evaluate CSRO code-policies against an opponent which plays this optimal Nash strategy at every hand of the repeated game. We also evaluate code-policies against two heuristic strategies with easy to detect patterns: *AlwaysCall* and *AlwaysFold* (a strategy that always folds when possible and otherwise calls).

Given an evaluation population of heuristic strategies (bots), we can adopt the three metrics introduced in (Lanctot et al., 2023). We denote the population of bots as $\mathcal{P}$.

Population Return is the average return of π against the population. This measures the generalization capability of agents against an unseen opponent at test-time:

$$\textsc{PopReturn}(\pi) = \mathbb{E}_{\pi' \sim \mathcal{P}}[u(\pi, \pi')]$$

Within Population Exploitability serves as an approximate exploitability metric:

$$\textsc{PopExpl}(\pi) = -\min_{\pi' \in \mathcal{P}} u(\pi, \pi')$$

Aggregate Score measures a balance between average- and worst-case analysis:

$$\textsc{AggScore}(\pi) = \textsc{PopReturn}(\pi) - \textsc{PopExpl}(\pi)$$

An agent can have both high PopReturn and PopExpl by winning against most of the bots in the population but being particularly exploitable by one opponent. In this case AggScore is a better measure of generalization capability.

3.3. Baselines

To evaluate CSRO, we compare against a strong, conventional baseline from the multi-agent learning literature. We implement a standard PSRO algorithm where the best response oracle is a deep reinforcement learning agent. The agent's policy is represented by a deep Long Short-Term Memory (LSTM)-based network and is trained using the Importance Weighted Actor-Learner Architecture (IMPALA) algorithm (Espeholt et al., 2018). The recurrent nature of LSTM makes it suitable for our repeated, partially observable game settings. This serves as a canonical baseline, allowing for a direct comparison between our method, which generates oracle policies as explicit code, and the standard approach of optimizing a policy in a neural network's weight space. We refer to the Supplementary Materials A.1.1 for implementation details.

In RRPS, we further include a direct comparison with an *LLM-based agent* adapting the simple yet effective approach proposed in (Lanctot et al., 2023). The agent's architecture requires no task-specific training, instead relying directly on the sequence-prediction capabilities of a pre-trained LLM. A history of previous actions is provided as a textual prompt and the model generates the subsequent actions for both player and opponent. The agent plays the best response to the predicted opponent action. While the original work utilized Chinchilla (Hoffmann et al., 2022), our implementation substitutes this family of LLMs for the pre-trained Gemma 3 models (Team, 2025) to benchmark a more recent and accessible model on this task. See Table 6 for full results.

机器人种群代表了多种多样的RRPS策略，有助于衡量智能体的泛化能力和可利用度（Lanctot等人，2023年）。图1展示了CSRO在RRPS环境中的一次运行示例，呈现了收益和均衡解随迭代的演化，以及对抗机器人种群时的回报。

对于重复勒杜克扑克，我们使用反事实遗憾最小化＋（反事实遗憾最小化+）计算单局博弈的纳什均衡（Tammelin，2014年）。我们运行反事实遗憾最小化+，共进行10000次迭代，并使用加权平均策略。我们让 CSRO 代码策略与一个在重复博弈的每局手牌中都采用该最优纳什策略的对手对战，以进行评估。我们还让代码策略与两种具有易于识别模式的启发式策略对战：*AlwaysCall* 和*AlwaysFold* （一种只要可以弃牌就始终弃牌，否则跟注的策略）。

给定一个由启发式策略（机器人）构成的评估种群，我们可以采用以下文献中介绍的三个指标（Lanctot 等人，2023年）。我们将机器人种群记为$\mathcal{P}$。

种群回报是 π 对战该种群时的平均回报。该指标衡量智能体在测试时面对未见过的对手的泛化能力：

$$\textsc{PopReturn}(\pi) = \mathbb{E}_{\pi' \sim \mathcal{P}}[u(\pi, \pi')]$$

种群内可利用度用作近似的可利用度指标：

$$\textsc{PopExpl}(\pi) = -\min_{\pi' \in \mathcal{P}} u(\pi, \pi')$$

综合得分衡量平均情况分析与最坏情况分析之间的平衡：

$$\textsc{AggScore}(\pi) = \textsc{PopReturn}(\pi) - \textsc{PopExpl}(\pi)$$

一个智能体可能通过战胜种群中的大多数机器人，同时具有较高的种群回报和种群可利用度，但却特别容易被某一个对手利用。在这种情况下，综合得分是衡量泛化能力的更好指标。

3.3. 基线

为评估CSRO，我们将其与多智能体学习文献中一个强大的传统基线进行比较。我们实现了标准的策略空间响应预言机算法，其中最佳响应预言机是一个深度强化学习智能体。该智能体的策略由基于深度长短期记忆（LSTM）的网络表示，并使用重要性加权行动者-学习者架构（IMPALA）算法进行训练（Espeholt等人，2018年）。LSTM的循环特性使其适用于我们的重复、部分可观测博弈设置。这是一个典型基线，可用于直接比较我们以显式代码生成预言机策略的方法与在神经网络权重空间中优化策略的标准方法。实现细节请参阅补充材料A.1.1 。

在RRPS中，我们还与一个基于大语言模型的智能体进行了直接比较，该智能体采用了（Lanctot等人，2023年）提出的简单而有效的方法。该智能体的架构无需任务特定训练，而是直接依赖预训练大语言模型的序列预测能力。历史动作以文本提示的形式提供给模型，模型随后生成玩家和对手的后续动作。智能体针对预测的对手动作采取最优响应。原始工作使用了Chinchilla（Hoffmann等人，2022年），而我们的实现则将这一大语言模型系列替换为预训练的Gemma 3模型（Team，2025年），以在该任务上对更新且更易获取的模型进行基准评估。完整结果见表6。

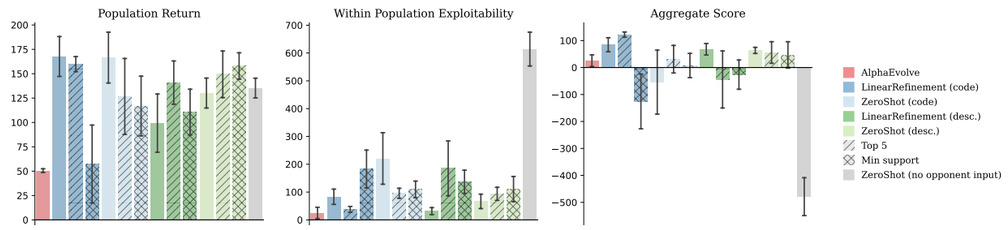

Figure 2 | Performance of CSRO variants in Repeated Rock-Paper-Scissors

In addition to the LLM agent based on Chinchilla, we include two further baselines from (Lanctot et al., 2023) that have not been trained against the evaluation population. Tabular Q-learning (QL) with recall length of 10 rounds and Contextual Regret Minimization (ContRM). For details we refer to previous work.

3.4. Implementation Details

We implement the CSRO oracle using the *Gemini 2.5 Pro* model (et al., 2025). All experiments are run for $K = 20$ iterations and $M = 10$ for *LinearRefinement*. For the *AlphaEvolve* oracle, we follow the method described in (Novikov et al., 2025; Romera-Paredes et al., 2024). Results are averaged over 5 seeds if not specified otherwise. The prompts and initial strategies are provided in Supplementary Materials A.2.

4. Results

4.1. Repeated Rock-Paper-Scissors

Method / Agent	PopReturn ↑	PopExpl ↓	AggScore ↑
CSRO			
- AlphaEvolve	50.5 ± 1.9	25.2 ± 20.3	25.4 ± 21.6
- LinearRef. (code)	159.8 ± 7.7	37.7 ± 10.6	122.1 ± 9.8
- LinearRef. (desc.)	99.3 ± 29.9	31.6 ± 12.4	67.7 ± 21.4
- ZeroShot (desc.)	130.2 ± 15.4	66.7 ± 25.9	63.5 ± 11.4
LLM Agent (27B)	193.2	67.2	126.0
PSRO-IMPALA	−108.9 ± 17.6	423.2 ± 28.0	−532.1 ± 41.5
QL (R=10)*	−0.5	8.6	−9.1
ContRM*	164.8	16.3	148.5
LLM Agent (70B)*	201.0	45.8	155.2

Table 1 | Performance in Repeated Rock-Paper-Scissors across *Population Return*, *Within Population Exploitability* and *Aggregate Score*. Results are averaged over 5 seeds. (* Denotes results reported in Lanctot et al. (2023).)

Our primary results in RRPS are summarized in Table 1. We report PopReturn (higher is better), PopExpl (lower is better), and AggScore (higher is better) for the meta-equilibrium strategies of CSRO variants at the final iteration ($K = 20$) as well as baseline performances. Our experiments show

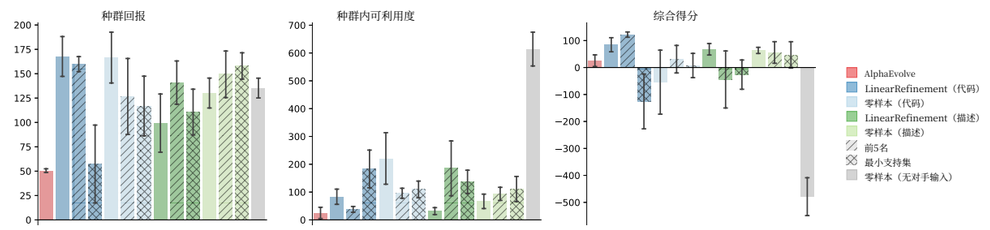

图 2 | CSRO 各变体在重复石头剪刀布博弈中的性能

除了基于Chinchilla的大语言模型智能体外，我们还纳入了（Lanctot等人，2023年）中的另外两个基线，它们均未针对评估种群进行训练：回溯长度为10回合的表格型Q学习（Q学习）和上下文遗憾最小化（ContRM）。详细信息请参阅先前工作。

3.4. 实现细节

我们使用 *Gemini 2.5Pro*模型实现 CSRO预言机（等人，2025年）。所有实验均运行 $K = 20$ 次迭代，而 $M = 10$ 次迭代用于 *LinearRefinement*。对于 *AlphaEvolve* 预言机，我们遵循以下文献所述的方法（Novikov 等人，2025年；Romera-Paredes 等人，2024年）。除非另有说明，结果均为使用 5 个随机种子所得结果的平均值。提示词和初始策略见补充材料 A.2。

4. 结果

4.1. 重复石头剪刀布博弈

方法 / 智能体	种群回报 ↑	种群可利用度 ↓	综合得分 ↑
CSRO			
- AlphaEvolve	50.5 ± 1.9	25.2 ± 20.3	25.4 ± 21.6
- LinearRef.（代码）	159.8 ± 7.7	37.7 ± 10.6	122.1 ± 9.8
- LinearRef.（描述）	99.3 ± 29.9	31.6 ± 12.4	67.7 ± 21.4
- 零样本（描述）	130.2 ± 15.4	66.7 ± 25.9	63.5 ± 11.4
大语言模型智能体（27B）	193.2	67.2	126.0
PSRO-IMPALA	−108.9 ± 17.6	423.2 ± 28.0	−532.1 ± 41.5
Q学习（R=10）*	−0.5	8.6	−9.1
ContRM*	164.8	16.3	148.5
大语言模型智能体（70B）*	201.0	45.8	155.2

表 1 | 重复石头剪刀布博弈中的性能，评估指标包括种群回报、种群内可利用度和综合得分。结果为5个随机种子的平均值。（*表示以下文献中报告的结果：Lanctot等人（2023年）。）

我们在RRPS中的主要结果汇总于表1。我们报告了种群回报（越高越好）、种群可利用度（越低越好）和综合得分（越高越好），这些指标针对最终迭代（$K = 20$）时各CSRO变体的元均衡策略，同时也报告了基线的表现。我们的实验表明

that among the proposed CSRO variants, the *AlphaEvolve* oracle is the most effective at generating a low-exploitability policy with a population exploitability of 25.2 ± 20.3. This outcome is consistent with the overall objective of the PSRO framework. The oracle generates policies that minimize worst-case performance against the current strategy mixture. As weaker opponents are assigned less support in the meta-game equilibrium, the oracle's objective naturally prioritizes robustness against strong strategies over maximizing exploitation of potential weak opponents. A powerful evolutionary process like AlphaEvolve is particularly well-suited to drive this optimization. *LinearRefinement (code)* with *Top 5* filtering achieves the highest aggregate score among our tested CSRO variants (122.1 ± 9.8). This performance is competitive with the strongest baseline LLM agent, a 27B parameter Gemma 3 model (126.0). We find that CSRO with intra-iteration policy refinement exhibits lower exploitability than the baseline Gemma 3 agents across all model sizes. Furthermore, all CSRO variants substantially outperform the PSRO-IMPALA baseline across all three metrics.

For context within the broader literature, (Lanctot et al., 2023) reported an impressive aggregate score of 155.2 for an LLM agent based on a Chinchilla model with 70B parameters, this is significantly higher than most baseline RL algorithms considered in their work. Among agents trained via self-play, ContRM, a contextual regret minimizer, achieved an aggregate score of 148.51. Their findings also underscore the importance of a balanced evaluation; for instance, while the Q-learning agent with recall 10 achieved very low exploitability (8.6), its performance was undermined by a near-zero PopReturn, resulting in an AggScore lower than that of a bot playing uniformly at random (`randbot`).

A detailed overview of results for all CSRO variants is shown in Figure 2 (and Supplementary Materials Table 7). The oracle's input format (code vs. description) interacts with the generation process. The benefit of using source code as input appears to be linked to iterative refinement. In the *ZeroShot* setting, providing opponent policies as natural language descriptions leads to better results; *ZeroShot (desc.)* achieves an aggregate score of 63.5 ± 11.4, compared to −54.3 ± 118.7 for *ZeroShot (code)*. A potential explanation is that generating a best response from a large set of source code in a single pass is a more complex task for which a textual summary provides a more tractable input. As a critical ablation, the *ZeroShot (no opponent input)* variant, which receives no information on opponent strategies, has a high PopReturn (135.3 ± 10.2) but also an extremely high PopExpl (614.2 ± 60.8), leading to a very poor aggregate score (−478.9 ± 70.2). This result underscores that opponent conditioning is the most important component of the prompt for generating robust strategies.

The *Top 5* filter generally yields better results than *Min support*. A plausible explanation is that the meta-game equilibrium often concentrates its probability mass on the most recent, single-best counter-policy. In such cases, *Min support* filtering would provide the LLM with only one opponent strategy as context. This can lead to overfitting, where the oracle generates a narrow policy that is highly effective against that one opponent but brittle against the wider, low-probability population. In contrast, the *Top 5* filter ensures a more diverse set of opponents is always present in the prompt, encouraging the LLM to generate more general and robust policies.

4.2. Repeated Leduc Hold'em Poker

In Table 2, we report CSRO performance in repeated Leduc poker as well as PSRO-IMPALA and CFR+ baselines, where metrics are computed with respect to the evaluation population $\mathcal{P}$ consisting of CFR+, *AlwaysCall* and *AlwaysFold*. Here we use description input and no filtering for all oracles, as this produced lowest exploitability in RRPS (see Supplementary Materials Table 7). Results show that the *AlphaEvolve* oracle achieves the highest PopReturn and AggScore, showing its efficacy of producing generalizable strategies. CSRO-AlphaEvolve also achieves an exploitability of 4.4 ± 0.6,

在所提出的CSRO变体中，*AlphaEvolve* 预言机在生成低可利用度策略方面最为有效，其种群可利用度为25.2 ± 20.3。这一结果与PSRO框架的总体目标一致。该预言机生成的策略旨在最小化面对当前混合策略时的最坏情况损失。由于较弱对手在元博弈均衡中被赋予的权重较低，预言机的目标自然会优先考虑对强策略的鲁棒性，而非最大限度地利用潜在的弱对手。像AlphaEvolve这样强大的演化过程尤其适合推动这一优化。采用前*5*名筛选的*LinearRefinement*（代码）在我们测试的CSRO变体中取得了最高的综合得分（122.1 ± 9.8）。这一表现可与最强的基线大语言模型智能体——拥有270亿参数的Gemma 3模型（126.0）相媲美。我们发现，在所有模型规模下，采用迭代内策略细化的CSRO的可利用度均低于基线Gemma 3智能体。此外，所有CSRO变体在全部三个指标上均显著优于PSRO-IMPALA基线。

从更广泛的文献背景来看，（Lanctot等人，2023年）报告称，一个基于700亿参数Chinchilla模型的大语言模型智能体取得了155.2的优异综合得分，显著高于其研究中考察的大多数基线强化学习算法。在通过自我博弈训练的智能体中，上下文遗憾最小化器ContRM取得了148.51的综合得分。他们的研究结果也凸显了均衡评估的重要性；例如，尽管回溯长度为10的Q学习智能体实现了极低的可利用度（8.6），但其接近零的种群回报拖累了整体表现，导致综合得分低于以均匀随机方式行动的机器人（randbot）。

所有CSRO变体的详细结果概览见图2（以及补充材料表7）。预言机的输入格式（代码或描述）与生成过程存在交互作用。使用源代码作为输入的优势似乎与迭代改进有关。在零样本设置下，以自然语言描述的形式提供对手策略会取得更好的结果；零样本（描述）的综合得分为63.5 ± 11.4，而零样本（代码）为 −54.3 ± 118.7。一种可能的解释是，单次处理大量源代码并生成最优响应是一项更复杂的任务，而文本摘要为此提供了更易处理的输入。作为一项关键消融实验，零样本（无对手输入）变体不接收任何有关对手策略的信息，其种群回报较高（135.3 ± 10.2），但种群可利用度也极高（614.2 ± 60.8），导致综合得分极差（−478.9 ± 70.2）。这一结果凸显出，对手条件化是用于生成鲁棒策略的提示词中最重要的组成部分。

前*5*名 筛选通常比最小支持集筛选效果更好。一个合理的解释是，元博弈均衡往往将其概率质量集中在最新的单个最优反制策略上。在这种情况下，最小支持集 筛选只会向大语言模型提供一种对手策略作为上下文。这可能导致过拟合，即预言机生成的策略适用范围狭窄，虽然对该对手非常有效，但面对更广泛的低概率策略群体时却十分脆弱。相比之下，前*5*名 筛选确保提示词中始终包含一组更多样化的对手，从而鼓励大语言模型生成更通用、更稳健的策略。

4.2. 重复勒杜克德州扑克

在表 2 中，我们报告了 CSRO 在重复勒杜克扑克中的性能，以及 PSRO-IMPALA 和反事实遗憾最小化+基线的性能，其中各项指标均相对于由反事实遗憾最小化+、*AlwaysCall* 和*AlwaysFold*组成的评估种群 P 计算。这里，我们对所有预言机均使用描述输入，且不进行筛选，因为这一配置在 RRPS 中产生了最低的可利用度（见补充材料表 7）。结果表明，*AlphaEvolve* 预言机取得了最高的种群回报和综合得分，展示了其生成可泛化策略的有效性。CSRO-AlphaEvolve 还实现了 4.4 ± 0.6 的可利用度，

Method	PopReturn ↑	PopExpl ↓	AggScore ↑
CSRO			
- AlphaEvolve	49.3 ± 3.7	4.4 ± 0.6	44.9 ± 4.1
- LinearRefinement	43.8 ± 2.5	9.8 ± 3.0	34.0 ± 4.3
- ZeroShot	40.4 ± 1.6	19.6 ± 2.1	20.7 ± 3.0
PSRO-IMPALA	13.3 ± 6.9	58.4 ± 3.3	−45.0 ± 10.1
CFR+	39.8 ± 0.3	0.0 ± 0.0	39.8 ± 0.3

Table 2 | Performance in repeated Leduc hold'em. Results are averaged over 3 seeds.

competitive with CFR+, demonstrating the robustness of the meta-equilibrium found. We found that the bots' worst-case performances (PopExpl) consistently arose from matches against the CFR+ opponent. Both the average-case (PopReturn) and worst-case (PopExpl) performances of the CSRO variants directly correlates with the strength of the best-response oracle: *AlphaEvolve*, followed by *LinearRefinement* and *ZeroShot*. This finding underscores the critical role of a powerful best-response solver in achieving game-theoretically near-optimal policies in repeated Leduc Poker within the CSRO framework.

We present a detailed breakdown of returns against each opponent in the eval population in Supplementary Materials Table 8. Against *AlwaysCall*, CSRO-AlphaEvolve achieves an average return of 110.3 ± 9.7 by developing a powerful value betting strategy (see Section 4.3). This performance is significantly higher than that of the PSRO-IMPALA (57.7 ± 3.3) and CFR+ (62.1 ± 0.8) baselines. This demonstrates a key advantage of the policies found by CSRO framework: CSRO does not just learn a low-exploitable strategy, but finds strategies that dynamically capitalize on the opponents' predictability over the course of the repeated game to achieve a super-Nash level of exploitation. *LinearRefinement* achieves a higher average return against *AlwaysFold* (57.3 ± 8.8) compared to *AlphaEvolve* (42.0 ± 3.2) and on par with CFR+ (57.4 ± 0.2). *AlphaEvolve* minimizes exploitability against CFR+ while exploiting *AlwaysCall* significantly more than CFR+. *LinearRefinement* finds strategies that lose more to CFR+, still exploiting *AlwaysCall* more than CFR+, and exploiting *AlwaysFold* as much as CFR+. In summary, we see evidence that the various CSRO oracles find different non-dominated strategies.

4.3. Qualitative Analysis

A key advantage of CSRO is the generation of interpretable policies, which allows for direct inspection of an agent's strategic logic. By analyzing the best programs from our experiments, we find they contain multiple strategic components, parameters and even heuristic value estimations.

4.3.1. Repeated Rock-Paper-Scissors

The best-performing strategy, generated by *LinearRefinement (no filter) - Top 5*, is a sophisticated ensemble agent achieving an impressive PopReturn of 238.3 (see Supplementary Materials Table 7) that would have placed it in between 2nd and 3rd place in the competition. Its well-commented source code allows for a direct analysis of its strategy, which is built on several key principles. First, the agent's strategy is a form of brute-force pattern matching, enabled by a comprehensive ensemble of 32 predictors. Rather than attempting to first deduce an opponent's high-level strategy (e.g., Markov player), the agent tests a wide array of hypotheses in parallel on every single turn. This large portfolio—which includes high-order Markov models, reactive models, and numerous heuristic

方法	种群回报 ↑	种群可利用度 ↓	综合得分 ↑
CSRO			
- AlphaEvolve	49.3 ± 3.7	4.4 ± 0.6	44.9 ± 4.1
- LinearRefinement	43.8 ± 2.5	9.8 ± 3.0	34.0 ± 4.3
- 零样本	40.4 ± 1.6	19.6 ± 2.1	20.7 ± 3.0
PSRO-IMPALA	13.3 ± 6.9	58.4 ± 3.3	−45.0 ± 10.1
CFR+	39.8 ± 0.3	0.0 ± 0.0	39.8 ± 0.3

表 2 | 重复勒杜克扑克中的表现。结果取3个随机种子的平均值。

与反事实遗憾最小化+相当，证明了所找到的元均衡具有鲁棒性。我们发现，机器人最坏情况性能（种群可利用度）始终出现在与反事实遗憾最小化+对手的对局中。CSRO 各变体的平均情况（种群回报）和最坏情况（种群可利用度）性能均与最佳响应预言机的能力直接相关：*AlphaEvolve* 最强，其次是*LinearRefinement* 和零样本。这一发现凸显了强大的最佳响应求解器对于在 CSRO框架内实现重复勒杜克扑克的博弈论近似最优策略所起的关键作用。

我们在补充材料的表8中详细列出了对阵评估种群中各个对手的回报。对阵*AlwaysCall*时，CSRO-AlphaEvolve通过开发强大的价值下注策略（见第4.3节），实现了110.3 ± 9.7的平均回报。这一表现显著优于PSRO-IMPALA（57.7 ± 3.3）和反事实遗憾最小化+ （62.1 ± 0.8）基线。这展示了CSRO框架所找到的策略的一项关键优势：CSRO不仅学习低可利用性策略，还能找到在重复博弈过程中动态利用对手行为可预测性的策略，从而实现超纳什水平的利用。对阵*AlwaysFold* 时，*LinearRefinement* 获得的平均回报（57.3 ± 8.8）高于*AlphaEvolve* （42.0 ± 3.2），并与反事实遗憾最小化+ （57.4 ± 0.2）相当。*AlphaEvolve* 在对阵反事实遗憾最小化+ 时将可利用度降至最低，同时对*AlwaysCall* 的利用程度显著高于反事实遗憾最小化+。*LinearRefinement* 找到的策略在对阵反事实遗憾最小化+时损失更大，但对*AlwaysCall* 的利用程度仍高于反事实遗憾最小化+，对*AlwaysFold* 的利用程度则与反事实遗憾最小化+相当。总之，我们观察到的证据表明，各种CSRO预言机找到了不同的非支配策略。

4.3. 定性分析

CSRO 的一个关键优势是能够生成可解释策略，从而可以直接审视智能体的策略逻辑。通过分析实验中表现最佳的程序，我们发现这些程序包含多种策略组件、参数，甚至还有启发式价值估计。

4.3.1. 重复石头剪刀布博弈

表现最佳的策略由 *LinearRefinement*（无筛选）- 前*5*名 生成，是一种复杂的集成智能体，取得了令人瞩目的种群回报，达到 238.3 （见补充材料表 7），这一成绩足以使其在比赛中跻身第二名与第三名之间。其注释详尽的源代码支持对其策略进行直接分析，该策略建立在几个关键原则之上。首先，该智能体的策略是一种暴力模式匹配，由包含32个预测器的综合集成系统提供支持。该智能体并不尝试先推断对手的高层策略（例如，马尔可夫玩家），而是在每一轮中并行检验大量假设。这一庞大的组合包含高阶马尔可夫模型、反应式模型以及众多启发式

detectors—is designed on the principle that one of its specialized experts will likely find a strong correlation in any given opponent's behavior, relying on breadth rather than deduction. Second, it uses an aggressive, high-confidence adaptation mechanism. Expert votes are weighted by the fifth power of their score, allowing the agent to rapidly and decisively commit to a theory of the opponent's strategy based on recent success. This non-linear weighting enables fast adaptation in a competitive environment. Finally, its most advanced component is an explicit second-order "Theory of Mind" model. This component infers the opponent's likely predictive model by observing which of its own experts is currently most successful. It then simulates the opponent's prediction of its own move and plays the corresponding counter-move. See Supplementary Materials Listing 1 for the full source code.

4.3.2. Repeated Leduc hold'em poker

The policy that achieves the best PopReturn (77.8) and best AggScore (69.1) demonstrates an effective synthesis of opponent modeling to drive a classic Expected Value (EV) calculation. The EV is calculated based on two key estimations: the *equity* (the probability of winning at showdown) and a *statistical model* of the opponent's tendencies (e.g., the probability they will fold to a bet).

The agent combines these estimations to calculate the EV of a potential raise as a weighted average of two primary outcomes: the immediate profit from winning the pot if the opponent folds, versus the potential net gain or loss from playing for a larger pot at showdown if the opponent calls. This calculation explicitly weighs the risk and reward of bluffing versus betting for value.

The interpretability of this structure allows us to see precisely how it produces expert-level, adaptive strategies. For instance:

Against an *AlwaysCall* opponent, the bot learns the opponent's folding probability is near zero. Consequently, its EV calculation simplifies to a classic "value betting" strategy, where it ceases to bluff and only raises with very strong hands. Against an *AlwaysFold* opponent, the bot rapidly learns the folding probability is near 100%. Its EV calculation for a raise then becomes equal to the current pot size, making its own hand irrelevant. As a result, the agent learns to bluff relentlessly to exploit the opponent's passivity. This transparent adaptation, directly observable in the policy's code, demonstrates a level of strategic reasoning and interpretability that is fundamentally absent in opaque, black-box policies. See Supplementary Materials Listing 2 for the full source code.

The generated policies share a key characteristic across both games and multiple seeds: they are not monolithic but rather compositions of distinct, interpretable strategic modules. Crucially, policies often demonstrate a capacity for sophisticated opponent modeling. This demonstrates a capacity for higher-order reasoning that is transparent and verifiable, a feature that distinguishes CSRO-generated policies from their black-box counterparts.

5. Related Work

Our work builds directly upon the Policy-Space Response Oracles (PSRO) framework (Lanctot et al., 2017), a general-purpose algorithm for computing approximate Nash equilibria in large extensive-form games. Significant follow-up work has focused on improving the meta-solver, such as using alpha-rank (Omidshafiei et al., 2019) or developing more scalable training schemes like Pipeline PSRO (Mcaleer et al., 2020). While these advancements have improved convergence and scalability, they have largely retained the use of deep reinforcement learning as the core of the best response oracle, thereby inheriting its limitations in terms of sample complexity and policy opacity. CSRO is orthogonal to these meta-solver improvements; we instead focus on fundamentally changing the nature of the oracle itself to prioritize interpretability.

检测器，其设计原则是：面对任何给定的对手，其中某个专用专家很可能会在对手行为中发现强相关性，依靠的是覆盖广度而非演绎推理。其次，它采用了一种激进的高置信度自适应机制。专家投票按其得分的五次方进行加权，使智能体能够依据近期的成功表现，迅速而果断地采纳某种关于对手策略的假说。这种非线性加权使其能够在竞争环境中快速适应。最后，其最先进的组件是一个显式的二阶心智理论模型。该组件通过观察自身哪些专家当前最为成功，来推断对手可能采用的预测模型。随后，它模拟对手对自身行动的预测，并采取相应的反制行动。完整源代码见补充材料代码清单 1。

4.3.2. 重复勒杜克德州扑克

取得最高种群回报（77.8）和最高综合得分（69.1）的策略，有效地将对手建模融入了经典的期望值（EV）计算。期望值的计算基于两项关键估计：底池权益（在摊牌时获胜的概率），以及描述对手行为倾向的统计模型（例如，对手面对下注时弃牌的概率）。

智能体结合这些估计，将一次潜在加注的期望值计算为两种主要结果的加权平均值：若对手弃牌，则赢得底池并立即获利；若对手跟注，则在摊牌时争夺更大的底池，可能产生净收益或净损失。这一计算明确权衡了诈唬与价值下注各自的风险和回报。

可解释　　性使我们能够准确地看到这种结构如何产生专家级
，自适应策略。例如：

面对 *AlwaysCall* 对手时，机器人会学到对手的弃牌概率接近于零。因此，其期望价值计算简化为经典的“价值下注”策略，即不再诈唬，仅在手牌极强时加注。面对 *AlwaysFold* 对手时，机器人会迅速学到其弃牌概率接近 100%。此时，加注的期望价值便等于当前底池大小，自身手牌如何已无关紧要。因此，智能体学会了持续诈唬，以利用对手的被动性。这种透明的适应行为可以直接从策略代码中观察到，体现出不透明的黑箱策略根本不具备的策略推理水平和可解释性。完整源代码请参见补充材料中的代码清单 2 。

在两种博弈和多个随机种子下，生成策略都具有一个共同的关键特征：它们并非单一的整体，而是由不同的、可解释的策略模块组合而成。尤为重要的是，这些策略往往表现出复杂的对手建模能力。这体现出一种透明且可验证的高阶推理能力，也是CSRO生成的策略区别于黑箱策略的特征。

5. 相关工作

我们的工作直接建立在策略空间响应预言机（PSRO）框架（Lanctot等人，2017年）之上，这是一种用于计算大型扩展式博弈中近似纳什均衡的通用算法。大量后续工作致力于改进元求解器，例如使用AlphaRank（Omidshafiei等人，2019年），或开发更具可扩展性的训练方案，如流水线PSRO（Mcaleer等人，2020年）。尽管这些进展改善了收敛性和可扩展性，但它们基本上仍将深度强化学习作为最佳响应预言机的核心，因而也继承了其在样本复杂度和策略不透明性方面的局限。CSRO与这些元求解器改进相互独立；我们转而着重从根本上改变预言机本身的性质，将可解释性置于首位。

Another line of work focuses on applying game-theoretic solvers to guide strategic interactions of LLMs in dialogue-based games (Gemp et al., 2024). The authors formalize natural language negotiation and debate scenarios as extensive-form games. They integrate LLMs with solvers like CFR and PSRO to generate prompts containing high-level strategic instructions to the game-playing LLM, thereby steering its dialogue to be more rational and less exploitable. A key difference is that the best responses produced by their method are natural language prompts, whereas whereas CSRO outputs executable code. Previous work by Kempinski et al. (Kempinski et al., 2025) also explores using LLMs for iterative reasoning in game-theoretic settings. In their work, "Game of Thoughts," they propose a family of algorithms inspired by cognitive hierarchy theory, such as level-k reasoning, to iteratively refine an LLM's strategic play. Their methods, including a variant named Policy Space Response Oracle Language Model (PSROLM), similarly place an LLM within an "outer loop" to improve its performance against prior strategies. While both approaches validate the use of LLMs for sophisticated strategic reasoning, our CSRO framework is distinct in two fundamental ways. First, our primary contribution is the synthesis of explicit, human-readable *programmatic policies*. This focus on generating executable and commented code as the policy representation provides a level of interpretability and verifiability not present in methods that generate actions directly. Second, our algorithm incorporates a novel intra-iteration refinement loop, which tactically hardens a candidate policy against the current meta-game *before* adding it to the policy pool, ensuring a higher quality of strategies.

Our work is most closely related to (Bachrach et al., 2025) introducing LLM-PSRO. The authors focus on demonstrating the viability of LLMs as a code-generating, best-response oracle within a self-play loop. LLM-PSRO does not feature an inner feedback loop to iteratively refine the candidate policy. The work only includes inter-population evaluation and does not show performance against external populations or baselines.

6. Conclusion and Discussion

We introduced Code-Space Response Oracles (CSRO), a novel framework for finding approximate equilibria in multi-agent games. We addressed the critical limitation of policy opacity in standard methods like PSRO by replacing the conventional black-box, deep RL oracle with a Large Language Model. Our approach reframes the best response computation as a program synthesis task, prompting an LLM to generate policies directly as human-readable, executable code.

The results show that CSRO achieves competitive performance in terms of convergence to a low-exploitability equilibrium, particularly in complex games like multi-hand Leduc hold'em poker. In RRPS, while the AggScore of CSRO is below that of the best-performing LLM-based baseline reported in previous work, it is important to highlight the significant difference in computational efficiency. The baseline LLM agents are invoked on every turn, i.e., requiring 1000 model calls for a single game. In contrast, *LinearRefinement* generates a complete, reusable policy where the number of LLM calls only grows linearly with the number of iterations (e.g., $K = 20$ in our experiments). This demonstrates a favorable trade-off between peak performance and the computational cost of policy generation, positioning CSRO as a practical and efficient method for strategy discovery.

We acknowledge that the LLM's pre-training data likely contains knowledge of game strategies for a classic game like Rock-Paper-Scissors. However, the oracle must synthesize a novel best response to a specific, dynamically generated mixture of programmatic opponents provided in-context. The success of this process demonstrates a sophisticated capability for in-context strategic reasoning and code generation, not merely pattern retrieval.

Despite its promising results, our approach has several limitations. First, the performance of the

另一类工作着重将博弈论求解器用于指导大语言模型在基于对话的博弈中的策略交互（Gemp等人，2024年）。作者将自然语言谈判和辩论场景形式化为扩展式博弈。他们将大语言模型与反事实遗憾最小化、策略空间响应预言机等求解器相结合，生成包含高层策略指令的提示词，提供给参与博弈的大语言模型，从而引导其对话变得更加理性、更不易被利用。一个关键区别在于，他们的方法产生的最佳响应是自然语言提示词，而CSRO输出的是可执行代码。Kempinski等人此前的工作（Kempinski等人，2025年）也探索了使用大语言模型在博弈论情境下进行迭代推理。在其名为“思维博弈”的工作中，他们提出了一系列受认知层级理论启发的算法，例如k层级推理，以迭代方式改进大语言模型的博弈策略。他们的方法（包括一种名为策略空间响应预言机语言模型（策略空间响应预言机语言模型）的变体）同样将大语言模型置于“外循环”中，以提升其对抗先前策略时的表现。尽管这两类方法均验证了大语言模型用于复杂策略推理的可行性，但我们的CSRO框架在两个根本方面有所不同。首先，我们的主要贡献在于合成显式、人类可读的程序化策略。这种以生成可执行且带注释代码作为策略表示的做法，提供了直接生成动作的方法所不具备的可解释性和可验证性。其次，我们的算法引入了一种新颖的迭代内细化循环，针对当前元博弈从战术层面强化候选策略，然后才将其加入策略池，从而确保策略具有更高的质量。

与我们的工作最密切相关的是（Bachrach 等人，2025年）提出大语言模型策略空间响应预言机的研究。作者着重展示了在自博弈循环中，将大语言模型用作生成代码的最佳响应预言机的可行性。大语言模型策略空间响应预言机不具备用于迭代改进候选策略的内部反馈循环。该研究仅进行了种群间评估，并未展示其面对外部种群或基线时的性能。

6. 结论与讨论

我们提出了代码空间响应预言机（CSRO），这是一种用于寻找多智能体博弈中近似均衡的新框架。我们用大语言模型取代传统的黑箱深度强化学习预言机，解决了策略空间响应预言机等标准方法中策略不透明性这一关键局限。我们的方法将最优响应计算重新构建为程序合成任务，通过提示大语言模型，直接以人类可读的可执行代码形式生成策略。

结果表明，CSRO 在收敛至低可利用度均衡方面具有竞争力，尤其是在多手Leduc德州扑克等复杂博弈中。在 RRPS 中，尽管 CSRO 的综合得分低于先前研究中报告的表现最佳的基于大语言模型的基线，但值得强调的是，两者在计算效率上存在显著差异。基线大语言模型智能体在每个回合都会被调用，即单局博弈需要进行 1000 次模型调用。相比之下，*LinearRefinement* 会生成一套完整的可复用策略，其大语言模型调用次数仅随迭代次数线性增长（例如，在我们的实验中为 $K = 20$ ）。这表明，该方法在峰值性能与策略生成的计算成本之间实现了良好的权衡，使 CSRO 成为一种实用且高效的策略发现方法。

我们承认，大语言模型的预训练数据很可能包含石头剪刀布这类经典游戏的博弈策略知识。然而，预言机必须针对上下文中提供的、动态生成的特定程序化对手混合体，合成一种全新的最优响应。这一过程的成功展现了高水平的上下文内策略推理和代码生成能力，而非仅仅是模式检索。

尽管取得了令人鼓舞的结果，我们的方法仍存在一些局限性。首先，性能方面，

CSRO oracle is inherently tied to the capabilities of the underlying LLM and the quality of the prompt. Poorly structured prompts or less capable models can lead to suboptimal or syntactically incorrect code, requiring error-handling and regeneration logic. Second, while our method reduces the high sample complexity associated with RL training, it introduces the computational cost of repeated LLM API calls, which can be significant. Finally, the scalability of CSRO to games with vast, high-dimensional observation spaces (e.g., Stratego or StarCraft) remains an open question. Representing complex state and opponent strategies within the context length of current LLMs remains a substantial engineering challenge.

This manuscript was created with assistance from Gemini 2.5 Pro. The tool was used for revision and editing to improve clarity. The authors have carefully reviewed and approved all changes.

CSRO预言机本质上受制于底层大语言模型的能力和提示词的质量。结构不佳的提示词或能力较弱的模型可能生成次优代码或语法错误代码，因此需要错误处理和重新生成逻辑。其次，虽然我们的方法降低了强化学习训练所需的高样本复杂度，但也引入了反复进行大语言模型API调用的计算成本，这一成本可能相当高昂。最后，CSRO能否扩展到具有庞大高维观测空间的博弈（例如Stratego或星际争霸），仍是一个尚待解决的问题。如何在当前大语言模型的上下文长度限制内表示复杂状态和对手策略，仍是一项重大的工程挑战。

本稿件在 Gemini 2.5Pro 的辅助下撰写。该工具用于修订和编辑，以提高清晰度。作者已仔细审阅并批准所有修改。

References

Y. Bachrach, E. Toledo, K. Hambardzumyan, D. Magka, M. Josifoski, M. Jiang, J. Foerster, R. Raileanu, T. Shavrina, N. Cancedda, A. Ruderman, K. Millican, A. Lupu, and R. Hazra. Combining code generating large language models and self-play to iteratively refine strategies in games. In *Thirty-Fourth International Joint Conference on Artificial Intelligence - Demo Track*, pages 10999–11003, 2025.

A. Bighashdel, Y. Wang, S. McAleer, R. Savani, and F. A. Oliehoek. Policy space response oracles: A survey. In *Thirty-Third International Joint Conference on Artificial Intelligence - Survey Track*, pages 7951–7961, 2024.

D. Billings. The first international roshambo programming competition. *ICGA Journal*, 23(1):42–50, 2000a.

D. Billings. The second international roshambo programming competition. https://icga.org/icga/games/roshambo/, 2000b.

P. S. Castro, N. Tomasev, A. Anand, N. Sharma, R. Mohanta, A. Dev, K. Perlin, S. Jain, K. Levin, N. Elteto, W. Dabney, A. Novikov, G. C. Turner, M. K. Eckstein, N. D. Daw, K. J. Miller, and K. Stachenfeld. Discovering symbolic cognitive models from human and animal behavior. In *Forty-Second International Conference on Machine Learning*, 2025.

L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Thirty-Fifth International Conference on Machine Learning*, volume 80, pages 1407–1416, 2018.

G. C. et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.

I. Gemp, R. Patel, Y. Bachrach, M. Lanctot, V. Dasagi, L. Marris, G. Piliouras, S. Liu, and K. Tuyls. Steering language models with game-theoretic solvers. In *Agentic Markets Workshop at ICML 2024*, 2024.

J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, O. Vinyals, J. W. Rae, and L. Sifre. Training compute-optimal large language models. In *Thirty-Sixth International Conference on Neural Information Processing Systems*, 2022.

B. Kempinski, I. Gemp, K. Larson, M. Lanctot, Y. Bachrach, and T. Kachman. Game of thoughts: Iterative reasoning in game-theoretic domains with large language models. In *Twenty-Fourth International Conference on Autonomous Agents and Multiagent Systems*, pages 1088–1098, 2025.

B. Knoll, D. Lu, and J. Burdge. Rock Paper Scissors Programming Competition. https://rpscontest.com/. Accessed: 2025-10-8.

M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Pérolat, D. Silver, and T. Graepel. A unified game-theoretic approach to multiagent reinforcement learning. In *Thirty-First International Conference on Neural Information Processing Systems*, page 4193–4206, 2017.

M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei, et al. Openspiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019.

参考文献

Y. Bachrach, E. Toledo, K. Hambardzumyan, D. Magka, M. Josifoski, M. Jiang, J. Foerster, R. Raileanu, T. Shavrina, N. Cancedda, A. Ruderman, K. Millican, A. Lupu 和 R. Hazra. 结合代码生成大语言模型与自我博弈，迭代改进博弈策略。载于第三十四届国际人工智能联合会议演示分会，第10999–11003页，2025年。A. Bighashdel, Y. Wang, S. McAleer, R. Savani 和 F. A. Oliehoek. 策略空间响应预言机：综述。载于第三十三届国际人工智能联合会议综述分会，第7951–7961页，2024年。D. Billings. 首届国际石头剪刀布编程竞赛。《*ICGA*期刊》，23(1):42–50，2000年a。D. Billings. 第二届国际石头剪刀布编程竞赛。https://icga.org/icga/games/roshambo/，2000年b。P. S. Castro, N. Tomasev, A. Anand, N. Sharma, R. Mohanta, A. Dev, K. Perlin, S. Jain, K. Levin, N. Elteto, W. Dabney, A. Novikov, G. C. Turner, M. K. Eckstein, N. D. Daw, K. J. Miller 和 K. Stachenfeld. 从人类与动物行为中发现符号认知模型。载于第四十二届国际机器学习会议，2025年。L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg 和 K. Kavukcuoglu. IMPALA：采用重要性加权行动者-学习者架构的可扩展分布式深度强化学习。载于第三十五届国际机器学习会议，第80卷，第1407–1416页，2018年。G. C. 等。Gemini 2.5：以高级推理、多模态、长上下文和下一代智能体能力推动前沿发展，2025年。I. Gemp, R. Patel, Y. Bachrach, M. Lanctot, V. Dasagi, L. Marris, G. Piliouras, S. Liu 和 K. Tuyls. 利用博弈论求解器引导语言模型。载于*ICML 2024*智能体市场研讨会，2024年。J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, O. Vinyals, J. W. Rae 和 L. Sifre. 训练计算最优的大语言模型。载于第三十六届神经信息处理系统大会，2022年。B. Kempinski, I. Gemp, K. Larson, M. Lanctot, Y. Bachrach 和 T. Kachman. 思维博弈：利用大语言模型在博弈论领域进行迭代推理。载于第二十四届自主智能体与多智能体系统国际会议，第1088–1098页，2025年。B. Knoll, D. Lu 和 J. Burdge. 石头剪刀布编程竞赛。https://rpscontest.com/。访问日期：2025年10月8日。M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Pérolat, D. Silver 和 T. Graepel. 多智能体强化学习的统一博弈论方法。载于第三十一届神经信息处理系统大会，第4193–4206页，2017年。M. Lanctot, E. Lockhart, J.-B. Lespiau, V. Zambaldi, S. Upadhyay, J. Pérolat, S. Srinivasan, F. Timbers, K. Tuyls, S. Omidshafiei 等。OpenSpiel：博弈中的强化学习框架。*arXiv*预印本 *arXiv:1908.09453*，2019年。

M. Lanctot, J. Schultz, N. Burch, M. O. Smith, D. Hennes, T. Anthony, and J. Perolat. Population-based evaluation in repeated rock-paper-scissors as a benchmark for multiagent reinforcement learning. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856.

S. Mcaleer, J. Lanier, R. Fox, and P. Baldi. Pipeline psro: A scalable approach for finding approximate nash equilibria in large games. In *Thrity-Fourth International Conference on Neural Information Processing Systems*, volume 34, pages 20238–20248. Curran Associates, Inc., 2020.

A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

S. Omidshafiei, C. Papadimitriou, G. Piliouras, K. Tuyls, M. Rowland, J.-B. Lespiau, W. M. Czarnecki, M. Lanctot, J. Perolat, and R. Munos. α-rank: Multi-agent evaluation by evolution. *Scientific reports*, 9(1):9937, 2019.

B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

Y. Shoham and K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, New York, NY, USA, 2008.

F. Southey, M. Bowling, B. Larson, C. Piccione, N. Burch, D. Billings, and C. Rayner. Bayes' bluff: Opponent modelling in poker. In *Twenty-First Conference on Uncertainty in Artificial Intelligence*, pages 550–557, 2005.

O. Tammelin. Solving large imperfect information games using cfr+, 2014. URL https://arxiv.org/abs/1407.5042.

G. Team. Gemma 3 technical report, 2025.

O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

M. Lanctot, J. Schultz, N. Burch, M. O. Smith, D. Hennes, T. Anthony 和 J. Perolat. 重复石头剪刀布博弈中基于种群的评估：多智能体强化学习的基准。*Transactions on Machine Learning Research*，2023年。国际标准连续出版物号 2835-8856。S. Mcaleer, J. Lanier, R. Fox 和 P. Baldi. 流水线PSRO：在大型博弈中寻找近似纳什均衡的可扩展方法。载于第三十四届神经信息处理系统国际会议，第34卷，第20238–20248页。Curran Associates, Inc.，2020年。A. Novikov, N. V˜u, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli 和 M. Balog. AlphaEvolve：用于科学与算法发现的编程智能体，2025年。网址：https://arxiv.org/abs/2506.13131。S. Omidshafiei, C. Papadimitriou, G. Piliouras, K. Tuyls, M. Rowland, J.-B. Lespiau, W. M. Czarnecki, M. Lanctot, J. Perolat 和 R. Munos. α-点数：通过演化进行多智能体评估。科学报告，9(1):9937，2019年。B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi 等。利用大语言模型进行程序搜索以取得数学发现。自然，625(7995):468–475，2024年。Y. Shoham 和 K. Leyton-Brown. 多智能体系统：算法、博弈论与逻辑基础。剑桥大学出版社，美国纽约州纽约，2008年。F. Southey, M. Bowling, B. Larson, C. Piccione, N. Burch, D. Billings 和 C. Rayner. 贝叶斯诈唬：扑克中的对手建模。载于第二十一届人工智能不确定性会议，第550–557页，2005年。O. Tammelin. 使用反事实遗憾最小化求解大型不完美信息博弈+，2014年。网址：https://arxiv.org/abs/1407.5042。G. Team. Gemma 3技术报告，2025年。O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik 等。利用多智能体强化学习在星际争霸II中达到宗师水平。自然，575(7782):350–354，2019年。

A. Supplementary materials

A.1. Hyperparameters

A.1.1. PSRO-IMPALA

For all of our experiments, we perform a hyperparameter sweep over learning rates, hidden layer sizes, unroll length, entropy cost, batch size, and max gradient norm (see Table 3). Best performing hyper parameters are shown in Table 4 and Table 5.

Hyperparameter	**Range**
learning rates	[0.0001, 0.001]
hidden layer sizes	{[256, 128], [512, 256]}
unroll length	[20, 40]
entropy cost	[0.001, 0.1]
batch size	[16, 32, 64]
max gradient norm	{1, 10, 40}

Table 3 | IMPALA hyperparameter sweep.

Hyperparameter	**Value**
learning rates	0.0001
hidden layer sizes	[256, 128]
unroll length	20
entropy cost	0.001
batch size	16
max gradient norm	40
observation representation	observation_tensor() in OpenSpiel (Lanctot et al., 2019)

Table 4 | Hyperparameters in RRPS.

Hyperparameter	**Value**
learning rates	0.001
hidden layer sizes	[512, 256]
unroll length	40
entropy cost	0.01
batch size	64
max gradient norm	40
observation representation	infostate_tensor() in OpenSpiel (Lanctot et al., 2019)

Table 5 | Hyperparameter for repeated Leduc hold'em poker.

A. 补充材料

A.1. 超参数

A.1.1. PSRO-IMPALA

在所有实验中，我们均对学习率、隐藏层大小、展开长度、熵代价、批量大小和最大梯度范数进行超参数扫描（见表3）。表现最佳的超参数见表4和表5。

超参数	**范围**
学习率	[0.0001, 0.001]
隐藏层大小	{[256, 128], [512, 256]}
展开长度	[20, 40]
熵代价	[0.001, 0.1]
批量大小	[16, 32, 64]
最大梯度范数	{1, 10, 40}

表 3 | IMPALA 超参数扫描。

超参数	**值**
学习率	0.0001
隐藏层大小	[256, 128]
展开长度	20
熵代价	0.001
批量大小	16
最大梯度范数	40
观测表示	观测_OpenSpiel中的tensor()（Lanctot等人，2019年）

表 4 | RRPS中的超参数。

超参数	**值**
学习率	0.001
隐藏层大小	[512, 256]
展开长度	40
熵代价	0.01
批量大小	64
最大梯度范数	40
观测表示	infostate_tensor()，位于 OpenSpiel 中（Lanctot等人，2019年）

表 5 | 重复勒杜克扑克的超参数。

	Gemma 3 (PT)			
Bot Names	270M	1B	4B	27B
actr_lag2_decay	-20.1	-40.2	-22.1	-18.7
adddriftbot2	68.1	39.4	83.7	65.4
addshiftbot3	120.5	134.2	186.6	199.7
antiflatbot	995.3	993.1	991.1	991.2
antirotnbot	53.4	49.8	65.9	51.6
biopic	-25.3	-44.3	-25.6	-35.2
boom	27.6	-1.4	4.4	-21.2
copybot	965.6	962.4	968.2	969.8
debruijn81	-34.3	-45.4	-14.1	-7.4
driftbot	90.7	27.3	100.4	78.0
flatbot3	5.6	99.1	127.7	137.4
foxtrotbot	15.0	47.6	33.9	12.6
freqbot2	608.1	863.0	917.0	936.5
granite	116.8	79.2	84.9	99.7
greenberg	-47.1	-84.1	-52.0	-29.3
halbot	5.1	-96.7	-87.8	-55.9
inocencio	522.1	405.5	293.2	568.8
iocainebot	-80.2	-131.2	-68.1	-35.6
marble	113.2	74.5	91.9	98.8
markov5	-34.2	-58.2	-7.2	-18.5
markovbails	-39.2	-39.6	-6.2	-21.2
mixed_strategy	48.2	1.2	37.6	26.6
mod1bot	-55.9	-60.5	-54.9	-50.8
multibot	287.0	177.8	265.2	253.0
peterbot	650.6	770.1	865.6	879.6
phasenbott	-85.2	-141.3	-64.8	-52.8
pibot	6.5	-36.0	-21.5	-28.5
piedra	33.6	22.5	40.6	40.6
predbot	46.6	-38.1	-27.5	-67.2
r226bot	197.2	354.1	394.7	391.8
randbot	6.4	7.4	5.3	-9.3
robertot	-17.1	-13.1	-11.8	-27.9
rockbot	998.4	998.4	995.6	995.0
rotatebot	978.0	980.9	988.8	997.5
russrocker4	-51.8	-71.5	-31.2	-38.9
shofar	-33.6	-44.2	-24.8	-34.8
sunCrazybot	248.0	313.7	371.6	397.9
sunNervebot	-24.2	-71.3	-30.7	-32.8
sweetrock	36.6	41.6	38.9	42.0
switchalot	67.0	106.9	111.9	125.2
switchbot	181.6	218.4	204.8	226.3
textbot	57.3	138.9	180.1	153.8
zq_move	181.2	107.8	151.8	155.9
PopReturn	167.0	162.7	187.2	193.2
PopExpl	85.2	141.3	87.8	67.2
AggScore	81.8	21.4	99.4	126.0

Table 6 | LLM agent performance against Repeated Rock Paper Scissors bot population. Returns averaged over 16 games.

	Gemma 3（预训练）			
机器人名称	270M	1B	4B	27B
actr_lag2_decay	-20.1	-40.2	-22.1	-18.7
adddriftbot2	68.1	39.4	83.7	65.4
addshiftbot3	120.5	134.2	186.6	199.7
antiflatbot	995.3	993.1	991.1	991.2
antirotnbot	53.4	49.8	65.9	51.6
biopic	-25.3	-44.3	-25.6	-35.2
boom	27.6	-1.4	4.4	-21.2
copybot	965.6	962.4	968.2	969.8
debruijn81	-34.3	-45.4	-14.1	-7.4
driftbot	90.7	27.3	100.4	78.0
flatbot3	5.6	99.1	127.7	137.4
foxtrotbot	15.0	47.6	33.9	12.6
freqbot2	608.1	863.0	917.0	936.5
granite	116.8	79.2	84.9	99.7
greenberg	-47.1	-84.1	-52.0	-29.3
halbot	5.1	-96.7	-87.8	-55.9
inocencio	522.1	405.5	293.2	568.8
iocainebot	-80.2	-131.2	-68.1	-35.6
marble	113.2	74.5	91.9	98.8
markov5	-34.2	-58.2	-7.2	-18.5
markovbails	-39.2	-39.6	-6.2	-21.2
混合_策略	48.2	1.2	37.6	26.6
mod1bot	-55.9	-60.5	-54.9	-50.8
multibot	287.0	177.8	265.2	253.0
peterbot	650.6	770.1	865.6	879.6
phasenbott	-85.2	-141.3	-64.8	-52.8
pibot	6.5	-36.0	-21.5	-28.5
piedra	33.6	22.5	40.6	40.6
predbot	46.6	-38.1	-27.5	-67.2
r226bot	197.2	354.1	394.7	391.8
randbot	6.4	7.4	5.3	-9.3
robertot	-17.1	-13.1	-11.8	-27.9
rockbot	998.4	998.4	995.6	995.0
rotatebot	978.0	980.9	988.8	997.5
russrocker4	-51.8	-71.5	-31.2	-38.9
羊角号	-33.6	-44.2	-24.8	-34.8
sunCrazybot	248.0	313.7	371.6	397.9
sunNervebot	-24.2	-71.3	-30.7	-32.8
sweetrock	36.6	41.6	38.9	42.0
switchalot	67.0	106.9	111.9	125.2
switchbot	181.6	218.4	204.8	226.3
textbot	57.3	138.9	180.1	153.8
zq_移动	181.2	107.8	151.8	155.9
种群回报	167.0	162.7	187.2	193.2
种群可利用度	85.2	141.3	87.8	67.2
综合得分	81.8	21.4	99.4	126.0

表 6 | 大语言模型智能体对战重复石头剪刀布博弈机器人种群的表现。回报取 16 场博弈的平均值。

Method	Input	Filter	PopReturn ↑ mean	PopReturn ↑ max	PopExpl ↓ mean	PopExpl ↓ min	AggScore ↑ mean	AggScore ↑ max
AlphaEvolve	Desc.	-	50.5 ± 1.9	54.2	25.2 ± 20.3	3.3	25.4 ± 21.6	47.6
LinearRefinement	Desc.	-	99.3 ± 29.9	171.8	31.6 ± 12.4	11.8	67.7 ± 21.4	128.1
LinearRefinement	Code	Top 5	159.8 ± 7.7	189.1	37.7 ± 10.6	8.8	122.1 ± 9.8	141.3
ZeroShot	Desc.	-	130.2 ± 15.4	177.8	66.7 ± 25.9	18.9	63.5 ± 11.4	86.9
LinearRefinement	Code	-	167.7 ± 20.4	238.3	83.2 ± 27.4	15.7	84.5 ± 26.1	148.1
ZeroShot	Desc.	Top 5	149.4 ± 23.9	207.3	93.5 ± 23.5	38.2	55.8 ± 40.0	155.9
ZeroShot	Code	Top 5	126.7 ± 39.0	190.0	95.8 ± 17.9	53.2	30.9 ± 51.0	111.7
ZeroShot	Code	Min support	116.9 ± 30.6	160.9	109.5 ± 30.1	24.8	7.4 ± 45.0	122.7
ZeroShot	Desc.	Min support	157.9 ± 13.5	204.5	111.0 ± 45.2	38.0	46.9 ± 48.9	133.1
LinearRefinement	Desc.	Min support	110.6 ± 23.5	185.9	137.0 ± 41.6	28.9	−26.4 ± 54.2	84.4
LinearRefinement	Code	Min support	57.3 ± 40.0	152.3	183.2 ± 67.7	54.2	−125.8 ± 101.6	98.1
LinearRefinement	Desc.	Top 5	140.9 ± 22.2	187.6	185.2 ± 98.5	67.1	−44.3 ± 105.7	110.5
ZeroShot	Code	-	166.6 ± 26.1	208.3	220.9 ± 92.7	95.8	−54.3 ± 118.7	112.6
ZeroShot	-	-	135.3 ± 10.2	173.2	614.2 ± 60.8	411.7	−478.9 ± 70.2	-238.5

Table 7 | Performance of CSRO variants in Repeated Rock-Paper-Scissors ranked by PopExpl. Metrics are averaged over 3 seeds for AlphaEvolve and 5 seeds for all other variants.

Method	Opponent *CFR+*	Opponent *AlwaysCall*	Opponent *AlwaysFold*
CSRO			
- AlphaEvolve	−4.4 ± 0.6	110.3 ± 9.7	42.0 ± 3.2
- LinearRefinement	−9.8 ± 3.0	83.8 ± 5.1	57.3 ± 8.8
- ZeroShot	−19.6 ± 2.1	99.7 ± 3.0	41.0 ± 6.1
PSRO-IMPALA	−58.4 ± 3.3	57.7 ± 3.3	40.7 ± 14.1
CFR+	0.0 ± 0.0	62.1 ± 0.8	57.4 ± 0.2

Table 8 | Performance of CSRO variants in repeated Leduc hold'em. Metrics are averaged over 3 seeds.

方法	输入过滤器		种群回报 ↑ mean	种群回报 ↑ max	种群可利用度 ↓ mean	种群可利用度 ↓ min	综合得分 ↑ mean	综合得分 ↑ max
AlphaEvolve	描述	-	50.5 ± 1.9	54.2	25.2 ± 20.3	3.3	25.4 ± 21.6	47.6
LinearRefinement	描述	-	99.3 ± 29.9	171.8	31.6 ± 12.4	11.8	67.7 ± 21.4	128.1
LinearRefinement	Code	前5名	159.8 ± 7.7	189.1	37.7 ± 10.6	8.8	122.1 ± 9.8	141.3
零样本	描述	-	130.2 ± 15.4	177.8	66.7 ± 25.9	18.9	63.5 ± 11.4	86.9
LinearRefinement	Code	-	167.7 ± 20.4	238.3	83.2 ± 27.4	15.7	84.5 ± 26.1	148.1
零样本	描述	前5名	149.4 ± 23.9	207.3	93.5 ± 23.5	38.2	55.8 ± 40.0	155.9
零样本	Code	前5名	126.7 ± 39.0	190.0	95.8 ± 17.9	53.2	30.9 ± 51.0	111.7
零样本	Code	最小支持集	116.9 ± 30.6	160.9	109.5 ± 30.1	24.8	7.4 ± 45.0	122.7
零样本	描述	最小支持集	157.9 ± 13.5	204.5	111.0 ± 45.2	38.0	46.9 ± 48.9	133.1
LinearRefinement	描述	最小支持集	110.6 ± 23.5	185.9	137.0 ± 41.6	28.9	−26.4 ± 54.2	84.4
LinearRefinement	Code	最小支持集	57.3 ± 40.0	152.3	183.2 ± 67.7	54.2	−125.8 ± 101.6	98.1
LinearRefinement	描述	前5名	140.9 ± 22.2	187.6	185.2 ± 98.5	67.1	−44.3 ± 105.7	110.5
零样本	Code	-	166.6 ± 26.1	208.3	220.9 ± 92.7	95.8	−54.3 ± 118.7	112.6
零样本	-	-	135.3 ± 10.2	173.2	614.2 ± 60.8	411.7	−478.9 ± 70.2	-238.5

表 7 | CSRO 各变体在重复石头剪刀布中的性能，按种群可利用度排序。AlphaEvolve 的指标取 3 个随机种子的平均值，其余所有变体的指标取 5 个随机种子的平均值。

方法	对手 *CFR+*	对手 *AlwaysCall*	对手 *AlwaysFold*
CSRO			
- AlphaEvolve	−4.4 ± 0.6	110.3 ± 9.7	42.0 ± 3.2
- LinearRefinement	−9.8 ± 3.0	83.8 ± 5.1	57.3 ± 8.8
- 零样本	−19.6 ± 2.1	99.7 ± 3.0	41.0 ± 6.1
PSRO-IMPALA	−58.4 ± 3.3	57.7 ± 3.3	40.7 ± 14.1
CFR+	0.0 ± 0.0	62.1 ± 0.8	57.4 ± 0.2

表 8 | 性能 CSRO 各变体在重复勒杜克扑克中的性能。各项指标为 3 个随机种子下的平均值 .

Listing 1 | A repeated Rock-Paper-Scissors bot that achieves best average performance against the opponent pool

```python
from collections import defaultdict
import random
from typing import Any, Callable, Dict, List, Optional, Tuple


class Agent:
  """An expert agent for Repeated Rock Paper Scissors, engineered to defeat
  a field of sophisticated, adaptive opponents.

  This agent's strategy is a significant enhancement of the 'ensemble of
  experts' paradigm, built on four core principles:

  1.  **Superset Expert Ensemble**: The agent employs a vast portfolio of
      predictive experts designed to be a strict superset of those used by its
      competitors. This includes high-order Markov models (up to 8th order),
      complex reactive/joint models, and a wide range of periodic and
      heuristic predictors. By having more 'lenses' to view the game, it can
      detect patterns that other agents cannot.

  2.  **Aggressive, High-Confidence Adaptation**: The agent uses a highly
      aggressive weighted voting system, weighting each expert's vote by the
      *fifth power* of its score. This allows the agent to rapidly and
      decisively lock onto successful predictive models while effectively
      eliminating noise from the dozens of other, less-successful experts.

  3.  **Advanced Meta-Prediction ('Theory of Mind')**: The agent not only models
      the opponent but also models itself being modeled by the opponent. Its
      most critical expert, `meta_imitation`, deduces the opponent's likely
      predictive model by observing which of its own strategies is currently
      most successful. It then simulates the opponent's prediction of its own
      move and plays the perfect counter to the opponent's anticipated play.

  4.  **Strategic Randomization**: To combat the meta-predictors of its
      opponents, the agent introduces randomness at key decision points. When
      any of its predictive models or the final vote tally results in a tie,
      it chooses randomly among the best options. This makes its own behavior
      significantly harder to predict and exploit.

  Combined with finely tuned hyperparameters for a 1000-round game (a
  long-memory decay rate of 0.985), this agent is designed to out-learn,
  out-adapt, and out-think its competition.
  """

  def __init__(self):
    """Initializes the agent's constants, models, and meta-strategy state."""
    # Game constants
    self.MOVES = ['ROCK', 'PAPER', 'SCISSORS']
    self.COUNTER_MOVES = {
        'ROCK': 'PAPER',
        'PAPER': 'SCISSORS',
        'SCISSORS': 'ROCK',
    }

    # Hyperparameters tuned for a 1000-round game against strong learners
    self.decay = 0.985  # Slower decay for longer memory and pattern stability
    self.vote_power = (
        5  # Use score^5 for extremely aggressive, high-confidence adaptation
    )

    # State is initialized in reset()
    self.reset()

  def reset(self) -> None:
    """Resets all agent state for the beginning of a new match."""
    self.my_history: List[str] = []
    self.opponent_history: List[str] = []

```


代码清单 1 | 在对手池中取得最佳平均表现的重复石头剪刀布博弈机器人

1 from collections import defaultdict 2 import random 3 from typing import Any, Callable, Dict, List, Optional, Tuple 4 5 6 class Agent: 7 """一个用于重复石头剪刀布游戏的专家智能体，专为击败 8 众多复杂的自适应对手而设计。 9 10 该智能体的策略显著增强了“专家集成”11 范式，建立在四项核心原则之上： 12 13 1. **超集专家集成**：该智能体采用庞大的 14 预测专家组合，旨在构成竞争对手所用专家集合的 15 真超集。其中包括高阶马尔可夫模型（最高八阶）、 16 复杂的反应式模型与联合模型，以及各种周期性预测器和 17 启发式预测器。通过更多观察游戏的“视角”，它能够 18 发现其他智能体无法察觉的模式。 19 20 2. **激进的高置信度自适应**：该智能体使用高度 21 激进的加权投票系统，以每位专家评分的^{22} *五次方*对其投票进行加权。这使智能体能够迅速而 23 果断地锁定成功的预测模型，同时有效 24 消除其他数十位表现较差的专家所产生的噪声。 25 26 3. **高级元预测（“心智理论”）**：该智能体不仅对 27 对手进行建模，还对“对手如何对自身建模”进行建模。其 28 最关键的专家“meta_imitation”通过观察自身当前 29 最成功的策略，推断对手可能采用的 30 预测模型。随后，它模拟对手对自身 31 动作的预测，并针对对手预期采取的动作作出完美克制。 32 33 4. **策略性随机化**：为对抗对手的 34 元预测器，该智能体在关键决策点引入随机性。当 35 任一预测模型或最终票数统计出现平局时， 36 它会从最佳选项中随机选择。这使其自身行为 37 更难被预测和利用。 38 39 结合针对 1000 回合游戏精细调优的超参数（40 长记忆衰减率为 0.985），该智能体旨在学习能力、 41 适应能力和思考能力上全面超越竞争对手。 42 """ 43 44 def __init__(self): 45 """初始化智能体的常量、模型和元策略状态。""" 46 # 游戏常量 47 self.MOVES = ['石头', '布', '剪刀']48 self.COUNTER_MOVES ={ 49 '石头': '布', 50 '布': '剪刀', 51 '剪刀': '石头', 52 } 53 54 # 针对与强大学习型对手进行的 1000 回合游戏调优的超参数 55 self.decay = 0.985 # 更慢的衰减带来更长的记忆和更稳定的模式 56 self.vote_power =(57 5 # 使用评分^5实现极其激进的高置信度自适应 58) 59 60 # 状态在 reset() 中初始化 61 self.reset() 62 63 def reset(self) ->None: 64 """在新一场比赛开始时重置智能体的所有状态。""" 65 self.my_history: List[str] = []66 self.opponent_history: List[str] = []67

```
    # --- Underlying predictive models (Opponent's patterns) ---
    self.opp_markov_models = {
        i: defaultdict(lambda: defaultdict(int)) for i in range(1, 9)
    }  # n=1 to 8
    self.reactive_model = defaultdict(lambda: defaultdict(int))
    self.joint_hist_model = defaultdict(lambda: defaultdict(int))
    self.freq_model = defaultdict(int)

    # --- Underlying predictive models (My own patterns for meta-experts) ---
    self.my_markov_models = {
        i: defaultdict(lambda: defaultdict(int)) for i in range(1, 9)
    }  # n=1 to 8
    self.my_reactive_model = defaultdict(lambda: defaultdict(int))
    self.my_joint_hist_model = defaultdict(lambda: defaultdict(int))

    # --- Meta-learning state (The Experts) ---
    self.experts: Dict[str, Callable[[], Optional[str]]] = {
        # Opponent-history-based predictors (Markov up to 8th order)
        **{
            f'opp_markov{i}': self._create_opp_markov_predictor(i)
            for i in range(1, 9)
        },
        # My-history-based predictors
        'reactive': self._predict_reactive,
        'joint_hist': self._predict_joint_hist,
        # Simple statistical predictors
        'frequentist': self._predict_frequentist,
        # Heuristic predictors
        'copy_opponent': self._predict_copy_opponent,
        'copy_me': self._predict_copy_me,
        'rotator': self._predict_rotator,
        'counter_rotator': self._predict_counter_rotator,
        # Periodic predictors (up to 12 rounds)
        **{
            f'periodic_{i}': self._create_periodic_predictor(i)
            for i in range(2, 13)
        },
        # Standard Meta-predictors (Theory of Mind Level 1)
        **{
            f'meta_my_markov{i}': self._create_meta_my_markov_predictor(i)
            for i in range(1, 4)
        },
        'meta_my_reaction': self._predict_meta_my_reaction,
        'meta_my_joint': self._predict_meta_my_joint,
        # Advanced Meta-predictor (Theory of Mind Level 2)
        'meta_imitation': self._predict_meta_imitation,
    }
    self.expert_scores = {name: 1.0 for name in self.experts}
    self.last_predictions: Dict[str, Optional[str]] = {
        name: None for name in self.experts
    }

# --- Update Methods ---

def _update_scores(self, actual_opponent_move: str) -> None:
    """Updates expert scores based on their last prediction's accuracy."""
    for name, prediction in self.last_predictions.items():
        self.expert_scores[name] *= self.decay
        if prediction == actual_opponent_move:
            self.expert_scores[name] += 1

def _update_models(self, my_action: str, opponent_action: str) -> None:
    """Updates all internal statistical models with the last round's data."""
    self.freq_model[opponent_action] += 1

    if self.my_history:
        self.reactive_model[self.my_history[-1]][opponent_action] += 1
        joint_key = (self.my_history[-1], self.opponent_history[-1])
        self.joint_hist_model[joint_key][opponent_action] += 1
        self.my_joint_hist_model[joint_key][my_action] += 1
```

68 # --- 底层预测模型（对手行为模式）--- 69 self.opp_markov_models =｛ 70 i: defaultdict(lambda: defaultdict(int)) for i in range(1, 9) 71 ｝# n=1至 8 72 self.reactive_model = defaultdict(lambda: defaultdict(int)) 73 self.joint_hist_model = defaultdict(lambda: defaultdict(int)) 74 self.freq_model =defaultdict(int) 75 76 # --- 底层预测模型（供元专家使用的自身行为模式）--- 77 self.my_markov_models = ｛ 78 i: defaultdict(lambda: defaultdict(int)) for i in range(1, 9) 79 ｝# n=1至 8 80 self.my_reactive_model =defaultdict(lambda: defaultdict(int)) 81 self.my_joint_hist_model =defaultdict(lambda: defaultdict(int)) 82 83 # --- 元学习状态（专家）--- 84 self.experts: Dict[str, Callable[[], Optional[str]]] = ｛ 85 # 基于对手历史的预测器（最高八阶马尔可夫模型）86 **{ 87 f’opp_markov{i}’: self._create_opp_markov_predictor(i) 88 for i in range(1, 9) 89 }, 90 # 基于自身历史的预测器 91 ’reactive’: self._predict_reactive, 92 ’joint_hist’: self._predict_joint_hist, 93 # 简单统计预测器 94 ’frequentist’: self._predict_frequentist, 95 # 启发式预测器 96 ’copy_opponent’: self._predict_copy_opponent, 97 ’copy_me’: self._predict_copy_me, 98 ’rotator’: self._predict_rotator, 99 ’counter_rotator’: self._predict_counter_rotator, 100 # 周期性预测器（最长 12 回合）101 **{ 102 f’periodic_{i}’: self._create_periodic_predictor(i) 103 for i in range(2, 13) 104 }, 105 # 标准元预测器（一级心智理论）106 **{ 107 f’meta_my_markov{i}’: self._create_meta_my_markov_predictor(i) 108 for i in range(1, 4) 109 }, 110 ’meta_my_reaction’: self._predict_meta_my_reaction, 111 ’meta_my_joint’: self._predict_meta_my_joint, 112 # 高级元预测器（二级心智理论）113 ’meta_imitation’: self._predict_meta_imitation, 114 ｝115 self.expert_scores ={name: 1.0 for name in self.experts} 116 self.last_predictions: Dict[str, Optional[str]] = ｛ 117 name: None for name in self.experts 118 ｝119 120 # --- 更新方法 --- 121 122 def_update_scores(self, actual_opponent_move: str) ->None: 123 """根据专家上一次的预测准确率更新专家评分。""" 124 for name, prediction in self.last_predictions.items(): 125 self.expert_scores[name] *=self.decay 126 if prediction == actual_opponent_move: 127 self.expert_scores[name] += 1128 129 def_update_models(self, my_action: str, opponent_action: str) ->None: 130 """使用上一回合的数据更新所有内部统计模型。""" 131 self.freq_model[opponent_action] += 1132 133 if self.my_history: 134 self.reactive_model[self.my_history[-1]][opponent_action] += 1135 joint_key =(self.my_history[-1],self.opponent_history[-1]) 136 self.joint_hist_model[joint_key][opponent_action] += 1137 self.my_joint_hist_model[joint_key][my_action] += 1

```
    if self.opponent_history:
      self.my_reactive_model[self.opponent_history[-1]][my_action] += 1

    for n in self.opp_markov_models:
      if len(self.opponent_history) >= n:
        key = tuple(self.opponent_history[-n:])
        self.opp_markov_models[n][key][opponent_action] += 1
      if len(self.my_history) >= n:
        key = tuple(self.my_history[-n:])
        self.my_markov_models[n][key][my_action] += 1

  # --- Expert Prediction Methods ---

  def _get_best_prediction(self, model: Dict, key: Any) -> Optional[str]:
    """Helper to get the most likely move from a model given a key.

    Crucially, it breaks ties randomly to make our agent less predictable.
    """
    predictions = model.get(key)
    if not predictions:
      return None
    max_val = max(predictions.values())
    best_moves = [move for move, val in predictions.items() if val == max_val]
    return random.choice(best_moves)

  def _create_opp_markov_predictor(self, n: int) -> Callable[[], Optional[str]]:
    """Factory for creating n-th order Markov model predictors for the opponent."""

    def predictor() -> Optional[str]:
      if len(self.opponent_history) < n:
        return None
      key = tuple(self.opponent_history[-n:])
      return self._get_best_prediction(self.opp_markov_models[n], key)

    return predictor

  def _predict_reactive(self) -> Optional[str]:
    if not self.my_history:
      return None
    return self._get_best_prediction(self.reactive_model, self.my_history[-1])

  def _predict_joint_hist(self) -> Optional[str]:
    if not self.my_history:
      return None
    key = (self.my_history[-1], self.opponent_history[-1])
    return self._get_best_prediction(self.joint_hist_model, key)

  def _predict_frequentist(self) -> Optional[str]:
    return self._get_best_prediction({'freq': self.freq_model}, 'freq')

  def _predict_copy_opponent(self) -> Optional[str]:
    return self.opponent_history[-1] if self.opponent_history else None

  def _predict_copy_me(self) -> Optional[str]:
    return self.my_history[-1] if self.my_history else None

  def _predict_rotator(self) -> Optional[str]:
    if not self.opponent_history:
      return None
    return self.COUNTER_MOVES.get(self.opponent_history[-1])

  def _predict_counter_rotator(self) -> Optional[str]:
    if not self.opponent_history:
      return None
    return {v: k for k, v in self.COUNTER_MOVES.items()}.get(
        self.opponent_history[-1]
    )

  def _create_periodic_predictor(self, n: int) -> Callable[[], Optional[str]]:
```

138 139 if self.opponent_history: 140 self.my_reactive_model [self.opponent_history [-1]] [my_action] += 1141 142 for n in self.opp_markov_models: 143 if len(self.opponent_history) >=n: 144 key = tuple(self.opponent_history [-n:]) 145 self.opp_markov_models [n] [key] [opponent_action] += 1146 if len(self.my_history) >=n: 147 key =tuple(self.my_history [-n:]) 148 self.my_markov_models [n] [key] [my_action] += 1149 150 # --- 专家预测方法 --- 151 152 def _get_best_prediction(self, model: Dict, key:Any) -> Optional [str]: 153 """根据给定的键从模型中获取最可能的动作的辅助函数。154 155 关键在于，当多个动作并列时，它会随机选择，使我们的智能体更难被预测。156 """ 157 predictions =model.get(key) 158 if not predictions: 159 return None 160 max_val = max(predictions.values()) 161 best_moves = [move for move, val in predictions.items() if val ==max_val]162 return random.choice(best_moves) 163 164 def _ create_opp_markov_predictor(self, n: int) ->Callable [[], Optional [str]]: 165 """用于创建对手的n阶马尔可夫模型预测器的工厂。""" 166 167 def predictor() ->Optional [str]: 168 if len(self.opponent_history) <n: 169 return None 170 key =tuple(self.opponent_history [-n:]) 171 return self._get_best_prediction(self.opp_ markov_models [n] ,key) 172 173 return predictor 174 175 def_predict_reactive(self) ->Optional [str]: 176 if not self.my_history: 177 return None 178 return self._get_best_prediction(self.reactive_model, self.my_ history [-1]) 179 180 def_predict_joint_hist(self) ->Optional [str]: 181 if not self.my_history: 182 return None 183 key =(self.my_history [-1] ,self.opponent_history [-1]) 184 return self._get_best_ prediction(self.joint_hist_model, key) 185 186 def_predict_frequentist(self) ->Optional [str]: 187 return self._get_best_prediction({’ freq’ : self.freq_model}, ’ freq’) 188 189 def_predict_copy_ opponent(self) ->Optional [str]: 190 return self.opponent_history [-1] if self.opponent_history else None 191 192 def_predict_copy_me(self) ->Optional [str]: 193 return self.my_history [-1] if self.my_history else None 194 195 def_predict_rotator(self) ->Optional [str]: 196 if not self.opponent_history: 197 return None 198 return self.COUNTER_MOVES.get(self.opponent_history [-1]) 199 200 def_predict_counter_ rotator(self) ->Optional [str]: 201 if not self.opponent_history: 202 return None 203 return {v: k for k, v in self.COUNTER_MOVES.items()}.get(204 self.opponent_history [-1]205) 206 207 def_create_periodic_ predictor(self, n: int) ->Callable [[] ,Optional [str]]:

```
    """Factory for creating predictors for various periodicities."""

    def predictor() -> Optional[str]:
      return (
          self.opponent_history[-n] if len(self.opponent_history) >= n else None
      )

    return predictor

  def _create_meta_my_markov_predictor(
      self, n: int
  ) -> Callable[[], Optional[str]]:
    """Assumes opponent predicts my n-th order markov pattern and counters it."""

    def predictor() -> Optional[str]:
      if len(self.my_history) < n:
        return None
      key = tuple(self.my_history[-n:])
      predicted_my_move = self._get_best_prediction(
          self.my_markov_models[n], key
      )
      return (
          self.COUNTER_MOVES.get(predicted_my_move)
          if predicted_my_move
          else None
      )

    return predictor

  def _predict_meta_my_reaction(self) -> Optional[str]:
    """Assumes opponent predicts my reaction pattern and counters it."""
    if not self.opponent_history:
      return None
    predicted_my_reaction = self._get_best_prediction(
        self.my_reactive_model, self.opponent_history[-1]
    )
    return (
        self.COUNTER_MOVES.get(predicted_my_reaction)
        if predicted_my_reaction
        else None
    )

  def _predict_meta_my_joint(self) -> Optional[str]:
    """Assumes opponent predicts my joint history pattern and counters it."""
    if not self.my_history:
      return None
    key = (self.my_history[-1], self.opponent_history[-1])
    predicted_my_move = self._get_best_prediction(self.my_joint_hist_model, key)
    return (
        self.COUNTER_MOVES.get(predicted_my_move) if predicted_my_move else None
    )

  def _predict_meta_imitation(self) -> Optional[str]:
    """Predicts by assuming the opponent is using a model similar to our own

    best-performing model to predict our moves. This is Theory of Mind Level 2.
    """
    # Define the set of our experts that an opponent is likely to be using.
    opponent_modeling_experts = {
        'opp_markov1': (self.my_markov_models[1], self.my_history, 1),
        'opp_markov2': (self.my_markov_models[2], self.my_history, 2),
        'opp_markov3': (self.my_markov_models[3], self.my_history, 3),
        'reactive': (self.my_reactive_model, self.opponent_history, 1),
        'joint_hist': (
            self.my_joint_hist_model,
            (self.my_history, self.opponent_history),
            2,
        ),
    }
    # Find which of these opponent-modeling experts is performing best for us.
```

```
    best_expert_name = max(
        opponent_modeling_experts,
        key=lambda name: self.expert_scores.get(name, 0),
    )
    model, history_data, hist_len = opponent_modeling_experts[best_expert_name]

    key = None
    # Construct the key for the corresponding model of *our* behavior.
    if best_expert_name == 'joint_hist':
      if len(history_data[0]) >= 1 and len(history_data[1]) >= 1:
        key = (history_data[0][-1], history_data[1][-1])
    elif isinstance(history_data, list) and len(history_data) >= hist_len:
      key = tuple(history_data[-hist_len:])

    if key is None:
      return None

    # Predict our own move using the model the opponent is likely using.
    predicted_my_move = self._get_best_prediction(model, key)
    # Assume the opponent will play the counter to our predicted move.
    return (
        self.COUNTER_MOVES.get(predicted_my_move) if predicted_my_move else None
    )

# --- Main Agent Logic ---

def act(self, observation: dict[str, Any]) -> str:
  """Determines the agent's next move by learning from the past and using the

  weighted ensemble to predict the opponent's action.
  """
  # First round: reset state and play randomly.
  if observation.get('my_action') is None:
    self.reset()
    return random.choice(self.MOVES)

  # --- 1. Learning Phase (from previous turn's outcome) ---
  my_prev_action = observation['my_action']
  opp_prev_action = observation['opponent_action']

  self._update_scores(opp_prev_action)
  self._update_models(my_prev_action, opp_prev_action)
  self.my_history.append(my_prev_action)
  self.opponent_history.append(opp_prev_action)

  # --- 2. Prediction Phase (for the current turn) ---
  for name, func in self.experts.items():
    self.last_predictions[name] = func()

  # --- 3. Action Phase (for the current turn) ---
  vote_tally = defaultdict(float)
  for name, prediction in self.last_predictions.items():
    if prediction is not None:
      vote_tally[prediction] += self.expert_scores[name] ** self.vote_power

  if vote_tally:
    # Determine the most likely opponent move using our tie-breaking helper.
    predicted_opponent_move = self._get_best_prediction(
        {'vote': vote_tally}, 'vote'
    )
    return self.COUNTER_MOVES[predicted_opponent_move]

  # Ultimate fallback: If no expert could predict, play randomly.
  return random.choice(self.MOVES)
```

278 best_expert_name =max(279 opponent_modeling_experts, 280 key=lambda name: self.expert_scores.get(name, 0), 281) 282 model, history_data, hist_len = opponent_modeling_experts[best_expert_name]283 284 key =None 285 # 为*我方*行为的相应模型构造键。 286 if best_expert_name == 'joint_hist': 287 if len(history_data[0]) >= 1and len(history_data[1]) >= 1: 288 key = (history_data[0][-1],history_data[1][-1]) 289 elif isinstance(history_data, list) and len(history_data) >=hist_len: 290 key =tuple(history_data[-hist_len:]) 291 292 if key is None: 293 return None 294 295 # 使用对手可能采用的模型预测我方的动作。 296 predicted_my_move =self._get_best_prediction(model, key) 297 # 假设对手会采取克制我方预测动作的动作。 298 return (299 self.COUNTER_MOVES.get(predicted_my_move) if predicted_my_move else None 300) 301 302 # --- 智能体主逻辑 --- 303 304 def act(self, observation: dict[str, Any]) ->str: 305 """通过学习历史信息并使用 306 307 加权集成预测对手动作，确定智能体的下一个动作。 308 """ 309 # 第一轮：重置状态并随机采取动作。 310 if observation.get('my_action') is None: 311 self.reset() 312 return random.choice(self.MOVES) 313 314 # --- 1. 学习阶段（根据上一回合的结果）--- 315 my_prev_action =observation['my_action']316 opp_prev_action =observation['opponent_action']317 318 self._update_scores(opp_prev_action) 319 self._update_models(my_prev_action, opp_prev_action) 320 self.my_history.append(my_prev_action) 321 self.opponent_history.append(opp_prev_action) 322 323 # --- 2. 预测阶段（针对当前回合）--- 324 for name, func in self.experts.items(): 325 self.last_predictions[name] =func() 326 327 # --- 3. 行动阶段（针对当前回合）--- 328 vote_tally =defaultdict(float) 329 for name, prediction in self.last_predictions.items(): 330 if prediction is not None: 331 vote_tally[prediction] += self.expert_scores[name] **self.vote_power 332 333 if vote_tally: 334 # 使用平局裁决辅助函数确定对手最可能采取的动作。 335 predicted_opponent_move =self._get_best_prediction(336 {'vote': vote_tally}, 'vote' 337) 338 return self.COUNTER_MOVES[predicted_opponent_move]339 340 # 最终后备方案：如果没有专家能够做出预测，则随机采取动作。 341 return random.choice(self.MOVES)

Listing 2 | A repeated Leduc Poker bot that achieves best PopReturn against the opponent pool

```python
from typing import Any, Dict


class RepeatedLeducPokerBot:
  """A bot using a non-stationary strategy.

  It collapses its strategy into a single, continuous 'bravery' parameter. This
  parameter adapts based on the opponent's observed passivity and aggression,
  creating a reactive, non-stationary strategy designed to be difficult for
  learning opponents to model.
  """

  def __init__(self):
    """Initializes the bot."""
    self.player_id: int = -1
    self.opponent_id: int = -1
    self.card_ranking = {'J': 1, 'Q': 2, 'K': 3}
    self.game_count = 0

    self.alpha = 1.0  # Laplace smoothing parameter.
    # Model: info_set_str -> {'FOLD': count, 'CALL': count, 'RAISE': count}
    self.opponent_model = {}

  def _get_info_set_key(self, round_name, public_card, round_history) -> str:
    """Creates a unique key for an information set."""
    public_card_str = public_card if public_card else ''

    action_str = ''.join(a['action'][0] for a in round_history)
    return f'{round_name}:{public_card_str}:{action_str}'

  def receive_outcome(self, obs: Dict[str, Any]):
    """Receives the game outcome and updates the opponent action model."""
    self.game_count += 1
    action_history = obs.get('action_history', {})

    for round_name in ['PREFLOP', 'POSTFLOP']:
      round_history = action_history.get(round_name, [])
      # We need to build the history incrementally to get the info set *before* each action
      temp_round_history = []
      for action_event in round_history:
        if action_event.get('player_id') == self.opponent_id:
          public_card = (
              obs['public_state'].get('public_card')
              if round_name == 'POSTFLOP'
              else None
          )
          info_set_key = self._get_info_set_key(
              round_name, public_card, temp_round_history
          )

          if info_set_key not in self.opponent_model:
            self.opponent_model[info_set_key] = {
                'FOLD': self.alpha,
                'CALL': self.alpha,
                'RAISE': self.alpha,
            }

          action = action_event.get('action')
          if action in self.opponent_model[info_set_key]:
            self.opponent_model[info_set_key][action] += 1

        temp_round_history.append(action_event)

  def restart(self, player_id: int):
    """Starts a new game, assigning player position and adapting 'bravery'.

    Args:
        player_id: The player ID (0 or 1) for the new game.
    """
```


代码清单 2 | 在对抗对手池时取得最高种群回报的重复勒杜克扑克机器人

1 from typing import Any, Dict 2 3 4 class RepeatedLeducPokerBot: 5 """使用非平稳策略的机器人。 6 7 它将自身策略归结为单个连续的“勇气”参数。该 8 参数根据观测到的对手被动性和激进性进行调整， 9 从而形成一种反应式非平稳策略，旨在让 10 具有学习能力的对手难以建模。 11 """ 12 13 def__init__(self): 14 """初始化机器人。""" 15 self.player_id: int =-1 16 self.opponent_id: int = -1 17 self.card_ranking ={'J': 1, 'Q': 2, 'K': 3} 18 self.game_count = 0 19 20 self.alpha = 1.0 # 拉普拉斯平滑参数。 21 # 模型：info_set_str -> {'FOLD': count, 'CALL': count, 'RAISE': count} 22 self.opponent_model ={} 23 24 def_get_info_set_key(self, round_name, public_card, round_history) -> str: 25 """为信息集创建唯一键。""" 26 public_card_str =public_card if public_card else '' 27 28 action_str ='' .join(a['action'][0]for a in round_history) 29 return f'{round_name}:{public_card_str}:{action_str}' 30 31 def receive_outcome(self, obs: Dict[str, Any]): 32 """接收对局结果并更新对手行动模型。""" 33 self.game_count += 1 34 action_history =obs.get('action_history', {}) 35 36 for round_name in ['PREFLOP', 'POSTFLOP']: 37 round_history =action_history.get(round_name, []) 38 # 我们需要逐步构建历史记录，以获取每个动作*之前*的信息集 39 temp_round_history = []40 for action_event in round_history: 41 if action_event.get('player_id') == self.opponent_id: 42 public_card =(43 obs['public_state'].get('public_card') 44 if round_name =='POSTFLOP' _45 else None 46) 47 info_set_key =self._get_info_set_key(48 round_name, public_card, temp_round_history 49) 50 51 if info_set_key not in self.opponent_model: 52 self.opponent_model[info_set_key] ={ 53 'FOLD': self.alpha, 54 'CALL': self.alpha, 55 'RAISE': self.alpha, 56 } 57 58 action =action_event.get('action') 59 if action in self.opponent_model[info_set_key]: 60 self.opponent_model[info_set_key][action] += 1 61 62 temp_round_history.append(action_event) 63 64 def restart(self, player_id: int): 65 """开始新对局，分配玩家位置并调整“勇气”。 66 67 参数： 68 player_id: 新对局的玩家标识符（0 或 1）。 69 """

```
    self.player_id = player_id
    self.opponent_id = 1 - player_id

    # This method is called at the start of a new game.
    # We can reset any per-game state here if needed.
    pass

  def act(self, obs: Dict[str, Any]) -> str:
    """Outputs an action based on Expected Value (EV) calculations."""
    legal_actions = obs['player_view']['legal_actions']
    if not legal_actions:
      return 'CALL'

    # 1. Calculate Equity
    my_hand = obs['player_view']['hand']
    public_card = obs['public_state']['public_card']
    equity = self._calculate_equity(my_hand, public_card, obs)  # Pass obs here

    # 2. Calculate EV for each legal action
    evs = {}
    for action in legal_actions:
      evs[action] = self._calculate_action_ev(action, obs, equity)

    # 3. Choose best action
    best_action = max(evs, key=evs.get)
    return best_action

  def _calculate_equity(
      self, my_hand: str, public_card: str or None, obs: Dict[str, Any]
  ) -> float:
    """Calculates the equity of our hand against a distribution of opponent hands,

    weighted by opponent's observed actions in the current hand.
    Equity is the probability of winning at showdown.
    """
    deck = ['J', 'J', 'Q', 'Q', 'K', 'K']
    deck.remove(my_hand)
    if public_card:
      if public_card in deck:
        if public_card in deck:  # Check membership before removing
          deck.remove(public_card)

    possible_opponent_hands = deck
    if not possible_opponent_hands:
      return 0.5  # No possible hands for opponent, return neutral equity

    opponent_hand_weights = {
        hand: 1.0 for hand in possible_opponent_hands
    }  # Start with uniform weights

    # --- Add Weighting based on History ---
    action_history = obs.get('action_history', {})
    preflop_history = action_history.get('PREFLOP', [])
    postflop_history = action_history.get('POSTFLOP', [])

    opponent_preflop_raises = 0
    opponent_preflop_calls = 0  # Calls implying >0 bet (heuristic)
    opponent_postflop_raises = 0
    opponent_postflop_calls = 0  # Calls implying >0 bet (heuristic)

    # Simplified action parsing (doesn't know exact bet amounts)
    temp_round_hist = []
    for action_event in preflop_history:
      if action_event['player_id'] == self.opponent_id:
        action = action_event['action']
        # Check if this action faces a bet based on prior actions
        facing_bet = (
            any(
                a['action'] == 'RAISE'
                for a in temp_round_hist
```

```
70 self.player_id =player_id 71 self.opponent_id = 1- player_id 72 73 # 此方法在新游戏开始时调用。
74 # 如有需要，可在此重置每局游戏的状态。 75 pass 76 77 def act(self, obs: Dict[str, Any]) ->str: 78 """根据期
望值（EV）计算结果输出动作。""" 79 legal_actions =obs['player_view']['legal_actions']80 if not
legal_actions: 81 return 'CALL' 82 83 # 1. 计算底池权益 84 my_hand = obs['player_view']['hand']
85 public_card =obs['public_state']['public_card']86 equity =self._calculate_equity(my_hand,
public_card, obs) # 在此传入 obs 87 88 # 2. 计算每个合法动作的期望值 89 evs ={} 90 for action in legal_
actions: 91 evs[action] =self._calculate_action_ev(action, obs, equity) 92 93 # 3. 选择最佳动作 94 best_
action =max(evs, key=evs.get) 95 return best action_96 97 def_calculate_equity( 98 self, my_hand: str,
public_card: str or None, obs: Dict[str, Any]99 ) ->float: 100 """计算我方手牌相对于对手手牌分布的底池权益，
101 102 该分布根据当前这局中观察到的对手动作加权。 103 底池权益是在摊牌时获胜的概率。 104 """ 105 deck
= ['J','J','Q','Q','K','K']106 deck.remove(my_hand) 107 if public_card: 108 if
public_card in deck: 109 if public_card in deck: # 移除前检查是否存在 110 deck.remove(public_card) 111
112 possible_opponent_hands =deck 113 if not possible_opponent_hands: 114 return 0.5 # 对手没有可能
的手牌，返回中性底池权益 115 116 opponent_hand_weights ={ 117 hand: 1.0 for hand in possible_
opponent_hands 118 } # 初始使用均匀权重 119 120 # --- 根据历史记录添加加权 --- 121 action_history =
obs.get('action_history',{}) 122 preflop_history = action_history.get('PREFLOP',[]) 123
postflop_history =action_history.get('POSTFLOP',[]) 124 125 opponent_preflop_raises = 0126
opponent_preflop_calls = 0 # 暗示存在 >0下注的跟注（启发式方法） 127 opponent_postflop_raises = 0
128 opponent_postflop_calls = 0 # 暗示存在 >0下注的跟注（启发式方法） 129 130 # 简化的动作解析（不
知道确切的下注金额） 131 temp_round_hist = []132 for action_event in preflop_history: 133 if action_
event['player_id'] ==self.opponent_id: 134 action =action_event['action']135 # 根据先前的动作检
查此动作是否面临下注 136 facing_bet =( 137 any( 138 a['action'] =='RAISE' 139 for a in temp_
round_hist
```

```
            if a['player_id'] == self.player_id
          )
          or self.player_id == 1
      )  # P2 faces BB

      if action == 'RAISE':
        opponent_preflop_raises += 1
      elif (
          action == 'CALL' and facing_bet
      ):  # Count calls that match a bet/raise
        opponent_preflop_calls += 1
    temp_round_hist.append(action_event)

  temp_round_hist = []
  # Base postflop bet state on whether preflop ended with aggression
  postflop_facing_bet = False  # Initial state for P1 postflop
  if preflop_history:
    last_preflop_action = preflop_history[-1]
    # If last preflop action was a raise (either ours or theirs that we called)
    if last_preflop_action['action'] == 'RAISE':
      postflop_facing_bet = (
          True  # Simplified: assumes aggression carries over
      )

  for action_event in postflop_history:
    if action_event['player_id'] == self.opponent_id:
      action = action_event['action']
      # Check if opponent faced a bet in this round before their action
      currently_facing_bet = any(
          a['action'] == 'RAISE'
          for a in temp_round_hist
          if a['player_id'] == self.player_id
      )

      if action == 'RAISE':
        opponent_postflop_raises += 1
        postflop_facing_bet = True  # Their raise makes us face a bet
      elif (
          action == 'CALL' and currently_facing_bet
      ):  # Count calls that match a bet/raise
        opponent_postflop_calls += 1
        postflop_facing_bet = False  # Call equalizes bets
      elif (
          action == 'CALL' and not currently_facing_bet
      ):  # Zero-cost call (check)
        postflop_facing_bet = False  # Check equalizes bets
    else:  # Our action
      if action_event['action'] == 'RAISE':
        postflop_facing_bet = True  # We raised, opponent faces bet
      elif action_event['action'] == 'CALL':  # Our check or call
        # If we called a raise, bets are equal. If we checked, bets are equal.
        postflop_facing_bet = False
    temp_round_hist.append(action_event)

  # Apply weights (Heuristics - Requires Tuning)
  if opponent_preflop_raises > 0:
    for hand in list(opponent_hand_weights.keys()):
      if hand == 'J':
        opponent_hand_weights[hand] *= 0.1  # Less likely J
      if hand == 'Q':
        opponent_hand_weights[hand] *= 0.7
      if hand == 'K':
        opponent_hand_weights[hand] *= 2.0  # More likely K
  elif opponent_preflop_calls > 0:  # Called preflop but didn't raise
    for hand in list(opponent_hand_weights.keys()):
      if hand == 'J':
        opponent_hand_weights[hand] *= 1.2
      if hand == 'K':
        opponent_hand_weights[hand] *= 0.5  # Less likely K
```



```
if a['player_id'] == self.player_id ) or self.player_id == 1) # P2 面临大盲注 if action == 'RAISE': opponent_preflop_raises += 1elif ( action == 'CALL' and facing_bet ): # 统计匹配下注或加注的跟注 opponent_preflop_calls += 1temp_round_hist.append(action_event) temp_round_hist = [ ]# 根据翻牌前是否以激进性动作结束来确定翻牌后的下注状态 postflop_facing_bet =False # P1 翻牌后的初始状态 if preflop_history: last_preflop_action = preflop_history[-1 ]# 如果翻牌前的最后一个动作是加注（我方加注，或对手加注后我方跟注） if last_preflop_action['action'] == 'RAISE': postflop_facing_bet = ( True # 简化处理：假设激进性会延续 ) for action_event in postflop_history: if action_event['player_id'] == self.opponent_id: action = action_event['action']# 检查对手在本轮采取动作前是否面临下注 currently_facing_bet = any( a['action'] == 'RAISE' for a in temp_round_hist if a['player_id'] == self.player_id ) if action == 'RAISE': opponent_postflop_raises += 1 postflop_facing_bet = True # 对手的加注使我方面临下注 elif ( action == 'CALL' and currently_facing_bet ): # 统计匹配下注或加注的跟注 opponent_postflop_calls += 1postflop_facing_bet =False # 跟注使双方下注持平 elif ( action == 'CALL' and not currently_facing_bet ): # 零成本跟注（过牌） postflop_facing_bet =False # 过牌使双方下注持平 else: # 我方的动作 if action_event['action'] == 'RAISE': postflop_facing_bet = True # 我方加注，对手面临下注 elif action_event['action'] =='CALL': # 我方过牌或跟注 # 如果我方跟注了加注，双方下注持平。如果我方过牌，双方下注持平。 postflop_facing_bet = False temp_round_hist.append(action_event) # 应用权重（启发式方法——需要调参） if opponent_preflop_raises > 0: for hand in list(opponent_hand_weights.keys()): if hand == 'J': opponent_hand_weights[hand] *= 0.1 # 持有 J 的可能性较低 if hand == 'Q': opponent_hand_weights[hand] *= 0.7 if hand == 'K': opponent_hand_weights[hand] *= 2.0 # 持有 K 的可能性较高 elif opponent_preflop_calls > 0: # 翻牌前跟注但未加注 for hand in list(opponent_hand_weights.keys()): if hand == 'J': opponent_hand_weights[hand] *= 1.2 if hand == 'K': opponent_hand_weights[hand] *= 0.5 # 持有 K 的可能性较低
```

```
    if public_card:
      if opponent_postflop_raises > 0:
        for hand in list(opponent_hand_weights.keys()):
          opp_rank_tuple = self._get_hand_rank(hand, public_card)
          is_pair = (
              opp_rank_tuple[0] >= 3 + self.card_ranking['J']
          )  # Check if it's any pair
          if not is_pair:
            opponent_hand_weights[
                hand
            ] *= 0.1  # Unlikely to raise postflop without a pair
          else:
            opponent_hand_weights[
                hand
            ] *= 2.0  # More likely to have a pair if raised

    # Normalize weights to form a probability distribution
    total_weight = sum(opponent_hand_weights.values())
    if (
        total_weight <= 1e-9
    ):  # Avoid division by zero / handle case where all weights -> 0
      # Reset to uniform if weights collapse
      opponent_hand_weights = {hand: 1.0 for hand in possible_opponent_hands}
      total_weight = sum(opponent_hand_weights.values())
      if total_weight <= 1e-9:
        return 0.5  # Still no weight, return neutral equity

    opponent_hand_probs = {
        hand: weight / total_weight
        for hand, weight in opponent_hand_weights.items()
    }
    # Filter out hands with zero probability for efficiency
    opponent_hand_probs = {
        h: p for h, p in opponent_hand_probs.items() if p > 1e-9
    }

    if not opponent_hand_probs:
      return 0.5  # Check again after filtering

    win, tie = 0.0, 0.0  # Use floats for weighted sum
    total_prob_considered = 0.0

    if public_card:  # Postflop Equity Calculation
      my_rank = self._get_hand_rank(my_hand, public_card)
      for opp_hand, prob in opponent_hand_probs.items():
        opp_rank = self._get_hand_rank(opp_hand, public_card)
        if my_rank > opp_rank:
          win += prob
        elif my_rank == opp_rank:
          tie += prob
        total_prob_considered += prob

      if total_prob_considered == 0:
        return 0.5
      # Weighted Equity = (Sum P(OppHand)*Win(OppHand) + 0.5 * Sum P(OppHand)*Tie(OppHand)) / Sum
          P(OppHand)
      # Since Sum P(OppHand) = total_prob_considered, this simplifies:
      return (win + 0.5 * tie) / total_prob_considered

    else:  # Preflop Equity Calculation (Average over public cards)
      total_weighted_outcomes = 0.0  # Sum of probabilities of paths considered

      # Need the deck *after* my card is removed, before considering opponent hand
      base_deck_for_opp = ['J', 'J', 'Q', 'Q', 'K', 'K']
      base_deck_for_opp.remove(my_hand)

      for opp_hand, opp_prob in opponent_hand_probs.items():
        # Create the deck for board cards given my_hand and opp_hand
        deck_for_board = list(base_deck_for_opp)
        if opp_hand not in deck_for_board:
```

210 if public_card: 211 if opponent_postflop_raises > 0: 212 for hand in list(opponent_hand_weights.keys()): 213 opp_rank_tuple =self._get_hand_rank(hand, public_card) 214 is_pair =(215 opp_rank_tuple[0] >= 3 +self.card_ranking['J'] 216) # 检查是否为任意对子 217 if not is_pair: 218 opponent_hand_weights[219 hand220] *= 0.1 # 没有对子时不太可能在翻牌后加注 221 else: 222 opponent_hand_weights[223 hand224] *= 2.0 # 若加注，则更有可能持有对子 225 226 # 将权重归一化，形成概率分布 227 total_weight =sum(opponent_hand_weights.values()) 228 if (229 total_weight <= 1e-9 230): # 避免除以零 / 处理所有权重均为 -> 0的情况 231 # 如果权重塌缩，则重置为均匀分布 232 opponent_hand_weights ={hand: 1.0 for hand in possible_opponent_hands} 233 total_weight = sum(opponent_hand_weights.values()) 234 if total_weight <= 1e-9: 235 return 0.5 # 仍无权重，返回中性底池权益 236 237 opponent_hand_probs = { 238 hand: weight / total_weight 239 for hand, weight in opponent_hand_weights.items() 240 } 241 # 筛除零概率手牌以提高效率 242 opponent_hand_probs = { 243 h: p for h, p in opponent_hand_probs.items() if p > 1e-9 244 } 245 246 if not opponent_hand_probs: 247 return 0.5 # 筛选后再次检查 248 249 win, tie = 0.0, 0.0 # 使用浮点数计算加权和 250 total_prob_considered = 0.0 251 252 if public_card: # 翻牌后底池权益计算 253 my_rank =self._get_hand_rank(my_hand, public_card) 254 for opp_hand, prob in opponent_hand_probs.items(): 255 opp_rank =self._get_hand_rank(opp_hand, public_card) 256 if my_rank >opp_rank: 257 win += prob 258 elif my_rank ==opp_rank: 259 tie +=prob 260 total_prob_considered +=prob 261 262 if total_prob_considered == 0: 263 return 0.5 264 # 加权底池权益 = (求和 P(对手手牌)*获胜(对手手牌) + 0.5 * 求和 P(对手手牌)*平局(对手手牌)) /

Sum P(OppHand) 265 # 由于 Sum P(OppHand) =total_prob_considered，可简化为： 266 return (win + 0.5 *tie) / total_prob_considered 267 268 else: # 翻牌前底池权益计算（对公共牌取平均）269 total_weighted_outcomes = 0.0 # 所考虑路径的概率之和 270 271 # 需要在移除我的牌之后、考虑对手手牌之前的牌堆 272 base_deck_for_opp = ['J','J','Q','Q','K','K'] 273 base_deck_for_opp.remove(my_hand) 274 275 for opp_hand, opp_prob in opponent_hand_probs.items(): 276 # 根据 my_hand 和 opp_hand 创建用于发公共牌的牌堆 277 deck_for_board =list(base_deck_for_opp) 278 if opp_hand not in deck_for_board:

```
        # This can happen if opponent_hand_weights included hands now impossible
        # due to filtering or deck state issues. Skip this opponent hand.
        continue
      deck_for_board.remove(opp_hand)

      if not deck_for_board:  # No possible board cards (e.g., K vs K preflop)
        # Rank based on private card only
        my_rank_priv = self.card_ranking[my_hand]
        opp_rank_priv = self.card_ranking[opp_hand]
        outcome_weight = (
            opp_prob  # Weight by probability of this opponent hand
        )
        if my_rank_priv > opp_rank_priv:
          win += outcome_weight
        elif my_rank_priv == opp_rank_priv:
          tie += outcome_weight
        total_weighted_outcomes += outcome_weight
        continue  # Skip board card loop for this opponent hand

      # Loop through possible board cards
      num_board_cards = len(deck_for_board)
      board_card_prob = (
          1.0 / num_board_cards
      )  # Uniform prob for each possible board card

      for board_card in deck_for_board:
        outcome_weight = (
            opp_prob * board_card_prob
        )  # Combined probability of this path

        my_rank_board = self._get_hand_rank(my_hand, board_card)
        opp_rank_board = self._get_hand_rank(opp_hand, board_card)

        if my_rank_board > opp_rank_board:
          win += outcome_weight
        elif my_rank_board == opp_rank_board:
          tie += outcome_weight
        total_weighted_outcomes += outcome_weight

      if total_weighted_outcomes == 0:
        return 0.5  # Default if something went wrong
      # Equity = (Weighted Wins + 0.5 * Weighted Ties) / Total Weight Considered
      # Note: total_weighted_outcomes should ideally sum close to 1.0 if all paths are covered.
      # Normalizing by total_weighted_outcomes accounts for any paths skipped (e.g., impossible
          opp hands)
      return (win + 0.5 * tie) / total_weighted_outcomes

  def _get_hand_rank(
      self, private_card: str, public_card: str or None
  ) -> tuple[int, int]:
    """Assigns a numeric rank tuple to a hand for comparison."""
    private_rank = self.card_ranking[private_card]
    if not public_card:
      # Pre-showdown, just rank by private card
      return (private_rank, 0)

    public_rank = self.card_ranking[public_card]
    # Postflop
    if private_rank == public_rank:  # Pair
      # Major rank for pairs (4,5,6) is higher than for high cards (1,2,3)
      return (3 + private_rank, 0)
    else:  # High card
      # Major rank is the highest card, minor rank is the kicker
      return (max(private_rank, public_rank), min(private_rank, public_rank))

  def _get_opponent_action_probs(self, obs: Dict[str, Any]) -> Dict[str, float]:
    """Gets the opponent's likely response probabilities to our RAISE from our model."""
    round_name = obs['public_state']['round']
    public_card = obs['public_state']['public_card']
```

```
        # 如果 opponent_hand_weights 包含了因筛选或牌堆状态问题而变得不可能的手牌，# 就可
        能发生这种情况。跳过此对手手牌。continuedeck_for_board.remove(opp_hand)

280 283 284 if not deck_for_board: # 没有可能的公共牌（例如，翻牌前 K 对 K） 285 # 仅根据私有牌确定点数 286 my_rank_
priv =self.card_ranking[my_hand]287 opp_rank_priv =self.card_ranking[opp_hand]288 outcome_weight =( 289 opp_
prob # 按此对手手牌的概率加权 290 ) 291 if my_rank_priv >opp_rank_priv: 292 win +=outcome_weight 293 elif my_
rank_priv ==opp_rank_priv: 294 tie +=outcome_weight 295 total_weighted_outcomes +=outcome_weight 296
continue # 对此对手手牌跳过公共牌循环 297 298 # 遍历可能的公共牌 299 num_board_cards =len(deck_for_board) 300
board_card_prob =( 301 1.0 / num_board_cards 302 ) # 每张可能的公共牌的概率均相同 303 304 for board_card in deck_for
board:_ _ _305 outcome_weight =( 306 opp_prob *board_card_prob 307 ) # 此路径的联合概率 308 309
my_rank_board = self._get_hand_rank(my_hand, board_card) 310 opp_rank_board = self._get_hand_rank(opp_
hand, board_card) 311 312 if my_rank_board >opp_rank_board: 313 win +=outcome_weight 314 # 损失对 ’ win’ 或 ’
tie’ 的贡献为 0 315 elif my_rank_board ==opp_rank_board: 316 tie +=outcome_weight 317 total_weighted_outcomes +=
outcome_weight 318 319 if total_weighted_outcomes == 0: 320 return 0.5 # 出错时的默认值 321 # 底池权益 =（加权获胜次
数 + 0.5 *加权平局次数) / 纳入计算的总权重 322 # 注意：如果覆盖了所有路径，total_weighted_outcomes 的总和理想情况下应
接近 1.0。 323 # 用 total_weighted_outcomes 归一化，以考虑所有被跳过的路径（例如，不可能的对手手牌） 324 return (win
+ 0.5 *tie) / total_weighted_outcomes 325 326 def_get_hand_rank( 327 self, private_card: str, public_card: str or
None 328 ) ->tuple[int, int]: 329 """为手牌分配一个数值牌力元组以进行比较。""" 330 private_rank =self.card_ranking[
private_card]331 if not public_card: 332 # 摊牌前，仅按私有牌确定点数 333 return (private_rank, 0) 334 335 public_rank =
self.card_ranking[public_card]336 # 翻牌后 337 if private_rank ==public_rank: # 对子 338 # 对子的主点数 (4,5,6) 高于高牌
的主点数 (1,2,3) 339 return (3 +private_rank, 0) 340 else: # 高牌 341 # 主点数为最高牌的点数，次点数为踢脚牌的点数 342
return (max(private_rank, public_rank), min(private_rank, public_rank)) 343 344 def_get_opponent_action_
probs(self, obs: Dict[str, Any]) ->Dict[str, float]: 345 """从我们的模型中获取对手面对我方加注时可能的响应概率。""" 346
round_name =obs[’public_state’][’round’]347 public_card =obs[’public_state’][’public_card’]
```

```
    current_history = obs['action_history'][round_name]
    # Simulate our raise to create the info set the opponent would face
    simulated_history = current_history + [
        {'player_id': self.player_id, 'action': 'RAISE'}
    ]
    info_set_key = self._get_info_set_key(
        round_name, public_card, simulated_history
    )

    if info_set_key in self.opponent_model:
      counts = self.opponent_model[info_set_key]
      total = sum(counts.values())
      if total > 0:
        return {action: count / total for action, count in counts.items()}

    # Default assumption if we have no data
    return {'FOLD': 0.33, 'CALL': 0.34, 'RAISE': 0.33}

  def _calculate_action_ev(
      self, action: str, obs: Dict[str, Any], equity: float
  ) -> float:
    """Calculates the Expected Value of a single action."""
    pot_size = obs['public_state']['pot_size']

    committed = [(100 - c) for c in obs['public_state']['chips']]
    my_committed = committed[self.player_id]
    opp_committed = committed[self.opponent_id]
    amount_to_call = opp_committed - my_committed

    round_name = obs['public_state']['round']
    raise_amount = 2 if round_name == 'PREFLOP' else 4

    if action == 'FOLD':
      return 0

    if action == 'CALL':
      pot_after_call = pot_size + amount_to_call
      return (equity * pot_after_call) - amount_to_call

    if action == 'RAISE':
      # EV(Raise) = P(Opponent Folds) * (Win Pot) + P(Opponent Calls) * (EV at Showdown)
      # We simplify by ignoring opponent re-raises.

      opp_probs = self._get_opponent_action_probs(obs)
      prob_opp_folds = opp_probs['FOLD']

      # What we win if opponent folds
      win_on_fold = pot_size

      # What happens if opponent calls our raise
      amount_to_raise = amount_to_call + raise_amount
      # Pot includes current pot, my raise, and opponent's call of the raise amount
      pot_after_opp_calls = pot_size + amount_to_raise + raise_amount

      value_if_called = (equity * pot_after_opp_calls) - amount_to_raise

      return (prob_opp_folds * win_on_fold) + (
          (1 - prob_opp_folds) * value_if_called
      )

    return -999  # Should not be reached
```

A.2. Prompts and Initial Strategies

A.2.1. Repeated Rock-Paper-Scissors

348 349 current_history =obs[’ action_history’][round_name]350 # 模拟我方加注，以构建对手将面临的信息集 351 simulated_history =current_history + [352 {’ player_id’ : self.player_id, ’ action’ : ’ RAISE’ } 353]354 info_set_key =self._get_info_set_key(355 round_name, public_card, simulated_history 356) 357 358 if info_set_key in self.opponent_model: 359 counts =self.opponent_model[info_set_key]360 total = sum(counts.values()) 361 if total > 0: 362 return {action: count / total for action, count in counts.items()} 363 364 # 无数据时的默认假设 365 return {’ FOLD’ : 0.33, ’ CALL’ : 0.34, ’ RAISE’ : 0.33} 366 367 def_calculate_action_ev(368 self, action: str, obs: Dict[str, Any] ,equity: float 369) ->float: 370 """"计算单个动作的期望值。""" 371 pot_size =obs[’ public_state’][’ pot_size’]372 373 committed = [(100 - c) for c in obs[’ public_state’][’ chips’]]374 my_committed =committed[self.player_id]375 opp_committed =committed[self.opponent_id]376 amount_to_call =opp_committed - my_committed 377 378 round_name =obs[’ public_state’][’ round’]379 raise amount = 2if round name ==’ PREFLOP’ else 4_ _380 381 if action ==’ FOLD’ : 382 return 0 383 384 if action ==’ CALL’ : 385 pot_after_call =pot_size +amount_to_call 386 return (equity *pot_after_call) - amount_to_call 387 388 if action ==’ RAISE’ : 389 # 期望值(加注) = 概率(对手弃牌) *(赢得的底池) + 概率(对手跟注) * (摊牌时的期望值) 390 # 为简化计算，我们忽略对手再加注的情况。 391 392 opp_probs =self._get_opponent_action_probs(obs) 393 prob_opp_folds =opp_probs[’ FOLD’]394 395 # 对手弃牌时我方赢得的金额 396 win_on_fold =pot_size 397 398 # 对手跟注我方加注时的情况 399 amount_to_raise = amount_to_call +raise_amount 400 # 底池包括当前底池、我方加注投入的金额，以及对手跟注加注额所投入的金额 401 pot_after_opp_calls =pot_size +amount_to_raise +raise_amount 402 403 value_if_called = (equity * pot_after_opp_calls) - amount_to_raise 404 405 return (prob_opp_folds *win_on_fold) + (406 (1 - prob_opp_folds) *value_if_called 407) 408 409 return -999 # 不应执行到此处

A.2. 提示词和初始策略

A.2.1. 重复石头剪刀布博弈

Listing 3 | Prompt for Repeated Rock-Paper-Scissors

```
Act as an expert Python programmer and game theory enthusiast.

Your task is to implement an agent class called `Agent` that represents a
player's strategy in a game of Repeated Rock Paper Scissors.

## Game Rules:

* The game is played for 1000 rounds.
* Your goal is to maximize your total number of wins over the 1000 rounds.
* The valid moves are strings: 'ROCK', 'PAPER', 'SCISSORS'.
* ROCK beats SCISSORS, SCISSORS beats PAPER, and PAPER beats ROCK.

## Class Signature:

Your implementation must align with the following class structure.

```python
class Agent:

  def act(self, observation: dict[str, Any]) -> str:
    ...
```

Input details:

The input `observation` is a dictionary with two keys representing the moves
played by the player and the opponent in the previous round. Defaults to `None`
for the first round.

```python
observation = {
  'my_action': 'ROCK',  # None for the first round.
  'opponent_action': 'PAPER',  # None for the first round.
}
```
```

A.2.2. Repeated Leduc Poker

Listing 4 | Prompt for Repeated Leduc Poker

```
Act as an expert in computer games, opponent modeling, algorithm design and multiagent learning.
    Your task is to iteratively improve the provided bot in repeated leduc poker. The bot will
    play leduc poker with an opponent for multiple games, where in each game their player
    positions are randomly permutated. The primary goal is to increase the scores on the provided
    evaluation metrics, where larger values are better.

Always adhere to best practices in Python coding.

# Rule of Leduc Poker
Leduc Poker is a simplified two-player poker game, ideal for AI research, that uses a small deck
    to focus on core poker concepts like betting strategy and imperfect information.

Here is a detailed breakdown of the rules to clarify legal moves. Note that in this implementation
    , the "Check" action is not available; players must use "Call" instead. A call may be zero-
    cost if there is no outstanding bet to match.

**1. Setup & Preliminaries**
*   **Players:** 2.
*   **Deck:** 6 cards (two Jacks, two Queens, two Kings).
*   **Blinds:** Before cards are dealt, mandatory bets are posted:
    *   Player 1 (P1) posts a **Small Blind** of 1 unit.
    *   Player 2 (P2) posts a **Big Blind** of 2 units.
*   **The Deal:** Each player receives one private card, face down.

**2. Core Betting Rules**
*   **Raise Sizing:** The amount to raise is fixed.
```


清单 3 | 重复石头剪刀布博弈的提示词

扮演一位精通 Python 编程的专家兼博弈论爱好者。

你的任务是实现一个名为 `Agent` 的智能体类，用于表示重复石头剪刀布博弈中的玩家策略。

博弈规则：

* 博弈共进行 1000 回合。
* 你的目标是在 1000 个回合中最大化获胜总次数。
* 有效动作是以下字符串：'石头'、'布'、'剪刀'。
* 石头胜剪刀，剪刀胜布，布胜石头。

类签名：

你的实现必须符合以下类结构。

```python
class Agent:

  def act(self, observation: dict[str, Any]) -> str:
    ...
```

输入详情：

输入 `observation` 是一个包含两个键的字典，分别表示玩家和对手在上一回合中采取的动作。在第一个回合中，默认值为 `None`。

```python
observation = {
  'my_action': '石头',  # 第一个回合中为 None。
  'opponent_action': '布',  # 第一个回合中为 None。
}
```

A.2.2. 重复勒杜克扑克

清单 4 | 重复勒杜克扑克的提示词

请扮演计算机游戏、对手建模、算法设计和多智能体学习领域的专家。你的任务是在重复勒杜克扑克中迭代改进所提供的机器人。该机器人将与一名对手进行多局 Leduc 扑克博弈，每局博弈中双方的玩家位置都会随机排列。主要目标是提高所提供的评估指标的得分，数值越大越好。

始终遵循 Python 编码的最佳实践。

Leduc 扑克规则

Leduc 扑克是一种简化的双人扑克游戏，非常适合人工智能研究。它使用较小的牌堆，重点关注下注策略和不完美信息等扑克核心概念。

以下是规则的详细说明，用于明确合法行动。请注意，在此实现中，“过牌”动作不可用；玩家必须改用“跟注”。如果没有需要匹配的待跟下注，跟注可以是零成本的。

1. 设置与预备知识
* **玩家：** 2。
* **牌堆：** 6张牌（两张J牌、两张Q牌、两张K牌）。
* **盲注：**发牌前，须下强制下注：
    * 玩家1（P1）下1单位的**小盲注**。
    * 玩家2（P2）下2单位的**大盲注**。
* **发牌：**每位玩家获得一张牌面朝下的私有牌。

2. 核心下注规则
* **加注尺度：**加注额固定。

```
    *   **Round 1:** The raise amount is **2 units**.
    *   **Round 2:** The raise amount is **4 units**.
*   **Total Betting Cap:** The total betting cap for each round is a maximum of **two raises**.
*   **Acting First:** Player 1 (the small blind) acts first in both betting rounds (pre-flop and
    post-flop).

**3. Round 1: Pre-Flop Betting**
This round occurs before the public card is revealed.

*   **P1's First Action:** P1 must act on P2's 2-unit Big Blind.
    *   **Fold:** Forfeit the 1-unit blind. P2 wins the pot.
    *   **Call:** Match the 2 units by putting in 1 more unit.
    *   **Raise:** Make a 2-unit raise, for a total of 4 units (P1 puts in 3 units). The total
        betting cap has been reached.
*   **P2's Action:**
    *   If P1 **called**, P2 can **Call** (a zero-cost action, as bets are equal) to end the round
        , or **Raise** (by putting in 2 more units to make it 4 total).
    *   If P1 **raised**, P2 can only **Call** (by putting in 2 more units) or **Fold**. The
        betting cap has been reached.
*   **P1's Second Action (if necessary):** If P1 called and P2 then raised, the action returns to
    P1. P1 can only **Call** (by putting in 2 more units) or **Fold**.

**4. The Flop: Public Card**
After Round 1 betting concludes, one public card is dealt face-up. This card is shared by both
    players.

**5. Round 2: Post-Flop Betting**
This round occurs after the flop. There are no blinds.

*   **P1's First Action:**
    *   **Call:** Make a zero-cost call to pass the turn (as there is no outstanding bet).
    *   **Raise:** Make a 4-unit raise.
*   **P2's Action:**
    *   If P1 **called** (at zero-cost), P2 can also **Call** (at zero-cost, ending the round) or
        **Raise** 4 units.
    *   If P1 **raised**, P2 can **Call** (matching the 4 units), **Raise** (by putting in another
         4 units, for a total bet of 8), or **Fold**. The total betting cap has been reached.
*   **Subsequent Actions:**
    *   If P2 **raised** (after P1's initial zero-cost call), the action returns to P1, who can **
        Call** (the 4 unit bet), **Raise** (to 8 total), or **Fold**. The total betting cap has
        been reached.
    *   If a player **raises**, the other player can only **Call** or **Fold**, as the betting cap
         has been reached.

**6. Showdown & Hand Ranking**
If neither player folds, a showdown occurs after Round 2 betting.

*   **Hand:** A player's hand is their private card combined with the public card.
*   **Hand Ranks (best to worst):**
    1.  **Pair:** Two cards of the same rank (e.g., J-J). Higher pairs beat lower pairs.
    2.  **High Card:** If no one has a pair, the player with the highest card wins (K > Q > J).
*   **Ties:** If both players have the same hand rank (e.g., both have a King-high), the pot is
    split.

**7. Winning**
A player wins the pot either by being the only one left after the other folds, or by having the
    best hand at showdown.


# Program Skeleton
You should design the bot according to the following APIs:

'''python
class RepeatedLeducPokerBot:

  def receive_outcome(self, obs: dict[str, Any]):
    """Receive game outcome of previous game."""

  def restart(self, player_id: int):
    """Start a new round of leduc poker with being assigned the player position player_id."""
```



```
* **第1轮：**加注额为**2单位**。* **第2轮：**加注额为**4单位**。* **总下注上限：**每轮的总下注上限为最多**两次加注**。* **首先行动：**玩家1（小盲注）在两个下注轮（翻牌前和翻牌后）中均首先行动。24**3. 第1轮：翻牌前下注** 26 本轮在公共牌亮出之前进行。27* **P1的首次动作：**P1必须针对P2的2单位大盲注采取动作。* **弃牌：**放弃已下的1单位盲注。P2赢得底池。* **跟注：**再投入1单位，补齐至2单位。* **加注：**加注2单位，使总下注达到4单位（P1投入3单位）。此时已达到总下注上限。* **P2的动作：**** 如果P1**跟注**，P2可以**跟注**（由于双方下注相等，这是零成本行动）以结束本轮，也可以**加注**（再投入2单位，使总下注达到4单位）。* 如果P1**加注**，P2只能**跟注**（再投入2单位）或**弃牌**。此时已达到下注上限。* **P1的第二次动作（如有必要）：**如果P1跟注后P2加注，则再次轮到P1行动。P1只能**跟注**（再投入2单位）或**弃牌**。36 **4. 翻牌：公共牌** 38 第1轮下注结束后，发出一张牌面朝上的公共牌。双方玩家共用这张牌。39**5. 第2轮：翻牌后下注** 41 本轮在翻牌后进行，不设盲注。42* **P1的首次动作：**** **跟注：**进行零成本跟注，将行动权交给对方（因为没有待跟下注）。* **加注：**加注4单位。* **P2的动作：**** 如果P1**跟注**（零成本），P2也可以**跟注**（零成本，结束本轮），或**加注** 4 单位。* 如果P1**加注**，P2可以**跟注**（补齐至4单位）、**加注**（再加4单位，使总下注达到8单位）或**弃牌**。此时已达到总下注上限。* **后续动作：**** 如果P2**加注**（在P1首次零成本跟注之后），则再次轮到P1行动；P1可以**跟注**（跟上4单位下注）、**加注**（使总下注达到8单位）或**弃牌**。此时已达到总下注上限。* 如果一位玩家**加注**，另一位玩家只能**跟注**或**弃牌**，因为已达到下注上限。52**6. 摊牌与牌型排名** 54 如果双方均未弃牌，则在第2轮下注结束后摊牌。55* **手牌：**玩家的手牌由其私有牌与公共牌组成。* **牌型排名（从强到弱）：** 58 1. **对子：**两张点数相同的牌（例如J-J）。点数较大的对子胜过点数较小的对子。59 2. **高牌：**如果双方都没有对子，则持有最大单牌的玩家获胜（K > Q > J）。* **平局：**如果双方玩家的牌型排名相同（例如双方都是K高牌），则平分底池。61 **7. 获胜** 63 玩家可以通过以下任一方式赢得底池：对方弃牌后成为唯一剩余的玩家，或在摊牌时拥有最强手牌。64 65 66 # 程序框架 67 你应按照以下应用程序编程接口设计机器人：68 69 ‘ ‘ ‘python 70 class RepeatedLeducPokerBot: 71 72 def receive_outcome(self, obs: dict [str, Any ]): 73 """接收上一局的对局结果。""" 74 75 def restart(self, player_id: int): 76 """开始新一回合的Leduc 扑克，并被分配到玩家位置player_id。"""
```

````
  def act(self, obs: dict[str, Any]) -> str:
    """Output an action given an observation.

      Args:
        obs: a JSON observation dictionary.
      Output:
        action, an action in {{'FOLD', 'CALL', 'RAISE'}}
    """
```

# Example observation format
The act method is called each time the agent need to make a decision during a game, given an observation in the format of JSON. Here are some examples of an observation:
{{'player_view': {{'player_id': 0, 'current_player': True, 'hand': 'K', 'legal_actions': ['CALL', 'RAISE']}}, 'public_state': {{'round': 'PREFLOP', 'chips': [99, 99], 'pot_size': 2, 'public_card': None}}, 'action_history': {{'PREFLOP': [], 'POSTFLOP': []}}, 'game_result': None}}

{{'player_view': {{'player_id': 1, 'current_player': True, 'hand': 'K', 'legal_actions': ['CALL', 'RAISE']}}, 'public_state': {{'round': 'POSTFLOP', 'chips': [99, 99], 'pot_size': 2, 'public_card': 'Q'}}, 'action_history': {{'PREFLOP': [{{'player_id': 0, 'action': 'CALL'}}, {{'player_id': 1, 'action': 'CALL'}}], 'POSTFLOP': [{{'player_id': 0, 'action': 'CALL'}}]}}}, 'game_result': None}}

The receive_outcome method is called at the end of each game. The agent receives an observation of the final state of the game to improve game play for future games with the same opponent. The observation here contains final payoff for each player, and players' hands if the last action is not FOLD. Some examples:

{{'player_view': {{'player_id': 0, 'current_player': False, 'hand': 'K', 'legal_actions': []}}, 'public_state': {{'round': 'PREFLOP', 'chips': [101, 99], 'pot_size': 0, 'public_card': None}}, 'action_history': {{'PREFLOP': [{{'player_id': 0, 'action': 'RAISE'}}, {{'player_id': 1, 'action': 'FOLD'}}], 'POSTFLOP': []}}, 'game_result': {{'outcome': 'FOLD', 'returns': [1, -1], 'showdown_hands': None}}}}


{{'player_view':{{'player_id': 0, 'current_player': False, 'hand': 'J', 'legal_actions': []}}, 'public_state': {{'round': 'POSTFLOP', 'chips': [95, 105], 'pot_size': 0, 'public_card': 'Q'}}, 'action_history': {{'PREFLOP': [{{'player_id': 0, 'action': 'RAISE'}}, {{'player_id': 1, 'action': 'CALL'}}], 'POSTFLOP': [{{'player_id': 0, 'action': 'CALL'}}, {{'player_id': 1, 'action': 'CALL'}}]}}}, 'game_result': {{'outcome': 'SHOWDOWN', 'returns': [-5, 5], 'showdown_hands': [{{'player_id': 0, 'hand': 'J'}}, {{'player_id': 1, 'hand': 'K'}}]}}}}

The restart method is called at the beginning of each game, where the agent is informed the player position it will act as in the next game.

legal_actions is a subset of {{'FOLD', 'CALL', 'RAISE'}}


# Current program
Here is the current program we are trying to improve (you will need to propose a modification to it below):

{code}

# Opponents
Here are the summary of opponent codes you are trying to beat:
{instances}

Try to reason about these opponents, and come up with a strategy that can exploit them in general.

# *SEARCH/REPLACE block* Rules:

Every *SEARCH/REPLACE block* must use this format:
1. The opening fence: ```python
2. The start of search block: <<<<<<< SEARCH
3. A contiguous chunk of up to 4 lines to search for in the existing source code
4. The dividing line: =======
````

77 78 def act(self, obs: dict[str, Any]) ->str: 79 """根据给定观测输出一个动作。80 81 参数：82 obs：一个JSON观测字典。83 输出：84 action，{{'弃牌', '跟注', '加注'}} 中的一个动作 85 """ 86 ‘‘‘87 88 # 观测格式示例 89 每当智能体在博弈中需要做出决策时，都会调用act方法，并向其提供JSON格式的观测。以下是一些观测示例：90 {{'player_view': {{'player_id': 0, 'current_player': True, 'hand': 'K', 'legal_actions': ['跟注', '加注']}, 'public_state': {{'round': '翻牌前', 'chips': [99, 99], 'pot_size': 2, 'public_card': None}}, 'action_history': {{'PREFLOP': [], 'POSTFLOP': []}}, 'game_result': None}} 91 92 {{'player_view': {{'player_id': 1, 'current_player': True, 'hand': 'K', 'legal_actions': ['跟注', '加注']}, 'public_state': {{'round': '翻牌后', 'chips': [99, 99], 'pot_size': 2, 'public_card': 'Q'}}, 'action_history': {{'PREFLOP': [{{'player_id': 0, 'action': '跟注'}}, {{'player_id': 1, 'action': '跟注'}}], 'POSTFLOP': [{{'player_id': 0, 'action': '跟注'}}]}}, 'game_result': None}} 93 94 每局博弈结束时都会调用receive_outcome方法。智能体会收到该局博弈的终局状态观测，以改进今后与同一对手进行博弈时的表现。此处的观测包含每位玩家的最终收益；如果最后一个动作不是弃牌，还会包含各玩家的手牌。以下是一些示例：95 96 {{'player_view': {{'player_id': 0, 'current_player': False, 'hand': 'K', 'legal_actions': ['跟注']}, 'public_state': {{'round': '翻牌前', 'chips': [101, 99], 'pot_size': 0, 'public_card': None}}, 'action_history': {{'PREFLOP': [{{'player_id': 0, 'action': '加注'}}, {{'player_id': 1, 'action': '弃牌'}}], 'POSTFLOP': [{{'player_id': 0, 'action': '跟注'}}]}}, 'game_result': {{'outcome': '弃牌', 'returns': [1, -1], 'showdown_hands': None}}}} 97 98 99 {{'player_view':{{'player_id': 0, 'current_player': False, 'hand': 'J', 'legal_actions': []}}, 'public_state': {{'round': '翻牌后', 'chips': [95, 105], 'pot_size': 0, 'public_card': 'Q'}}, 'action_history': {{'PREFLOP': [{{'player_id': 0, 'action': '加注'}}, {{'player_id': 1, 'action': '跟注'}}, {{'player_id': 0, 'action': '跟注'}}, {{'player_id': 1, 'action': '跟注'}}], 'POSTFLOP': [{{'player_id': 0, 'action': '跟注'}}, {{'player_id': 1, 'action': '跟注'}}]}}, 'game_result': {{'outcome': '摊牌', 'returns': [-5, 5], 'showdown_hands': [{{'player_id': 0, 'hand': 'J'}}, {{'player_id': 1, 'hand': 'K'}}]}}}} 100 101 每局博弈开始时都会调用restart方法，告知智能体它在下一局博弈中将扮演的玩家位置。102 103 legal_actions是{{'弃牌', '跟注', '加注'}}的子集 104 105 106 # 当前程序 107 以下是我们正在尝试改进的当前程序（你需要在下文提出对它的修改）： 108 109 {code} 110 111 # 对手 112 以下是你要击败的对手的代码摘要：113 {instances} 114 115 请尝试分析这些对手，并提出一种能够普遍利用其弱点的策略。116 117 # *搜索/替换块*规则：118 119 每个*搜索/替换块*都必须采用以下格式：120 1. 起始围栏： ‘‘‘python 121 2. 搜索块的开头：<<<<<<<SEARCH 122 3. 要在现有源代码中搜索的一段连续代码，最多4行 123 4. 分隔行： =======
````

```
5. The lines to replace into the source code
6. The end of the replace block: >>>>>>> REPLACE
7. The closing fence: ```

Every *SEARCH* section must *EXACTLY MATCH* the existing file content, character for character,
    including all comments, docstrings, etc.

*SEARCH/REPLACE* blocks will replace *all* matching occurrences.
Include enough lines to make the SEARCH blocks uniquely match the lines to change.

Keep *SEARCH/REPLACE* blocks concise.
Break large *SEARCH/REPLACE* blocks into a series of smaller blocks that each change a small
    portion of the file.
Include just the changing lines, and a few surrounding lines if needed for uniqueness.
Do not include long runs of unchanging lines in *SEARCH/REPLACE* blocks.

To move code within a file, use 2 *SEARCH/REPLACE* blocks: 1 to delete it from its current
    location, 1 to insert it in the new location.

Make sure that the changes you propose are consistent with each other. For example, if you refer
    to a new config variable somewhere, you should also propose a change to add that variable.

Example:
```python
<<<<<<< SEARCH
    return total_loss
=======
    # Add sparsity-promoting regularization to the loss.
    total_loss += self.hypers.l1_reg_weight * l1_reg

    return total_loss
{replace}
```
and
```python
<<<<<<< SEARCH
  return hyper.zipit([
=======
  return hyper.zipit([
      hyper.uniform('l1_reg_weight', hyper.interval(0.0, 0.01)),
{replace}
```

{lazy_prompt}
ONLY EVER RETURN CODE IN A *SEARCH/REPLACE BLOCK*!

Task
{task_instruction} {focus_sentence} {trigger_chain_of_thought}
Describe each change with a *SEARCH/REPLACE block*.
```

124 5. 要替换到源代码中的行 125 6. 替换块的结尾： >>>>>>> REPLACE 126 7. 结束围栏： ‘ ‘ ‘127 128 每个*搜索*部分都必须与现有文件内容逐字符*完全匹配*，包括所有注释、文档字符串等。129130 *搜索/替换*块将替换*所有*匹配项。131 请包含足够的行，使搜索块能够唯一匹配要修改的行。132 133 保持*搜索/替换*块简洁。134 将大型*搜索/替换*块拆分为一系列较小的块，每个块仅修改文件的一小部分。135 仅包含发生变化的行；如果需要确保匹配的唯一性，可附带少量上下文行。136 不要在*搜索/替换*块中包含大段未发生变化的行。137 138 要在文件内移动代码，请使用2个*搜索/替换*块：一个用于从当前位置删除代码，另一个用于将其插入新位置。139 140 请确保你提出的各项修改彼此一致。例如，如果你在某处引用了一个新的配置变量，就还应提出添加该变量的修改。141 142 示例：143 ‘ ‘ ‘python144 <<<<<<<SEARCH 145 return total loss_146 =======147 # 为损失添加稀疏性促进正则化。148 total_loss += self.hypers.l1_reg_weight * l1_reg 149 150 return total loss_151 {replace} 152 ‘ ‘ ‘153 以及 154 ‘ ‘ ‘ python155 <<<<<<<SEARCH 156 return hyper.zipit([157 =======158 return hyper.zipit([159 hyper.uniform('l1_reg_weight', hyper.interval(0.0, 0.01)), 160 {replace} 161 ‘ ‘ ‘162 163 {lazy_prompt} 164 只能以*搜索/替换块*的形式返回代码！165 166 # 任务 167 {task_instruction} {focus_sentence} {trigger_chain_of_thought} 168 用一个*搜索/替换块*描述每项修改。

The initial strategy is a heuristic strategy by zero-shot prompting "give a strong heuristic repeated Leduc Poker strategy":

初始策略是通过零样本提示“给出一个强大的重复勒杜克扑克启发式策略”生成的启发式策略：

A.2.3. Repeated Leduc Poker

A.2.3. 重复勒杜克扑克

Listing 5 | Prompt for Repeated Leduc Poker

清单 5 | 重复勒杜克扑克的提示词

```python
import random
from typing import Any, Dict


class RepeatedLeducPokerBot:
  """
  A simple heuristic bot for repeated Leduc Poker.

  The bot's strategy is based on a set of rules that consider the strength
  of its private card, the public card (if revealed), and the current
```

```python
import random, from typing import Any, Dict class RepeatedLeducPokerBot: """ 一个用于重复勒杜克扑克的简单启发式机器人。该机器人的策略基于一组规则，这些规则考虑其私有牌的强度 、公共牌（如果已揭示）以及当前的
```

```
  betting round. It does not learn from the opponent's behavior.
  """

  def __init__(self):
    """Initializes the bot."""
    self.player_id: int = -1
    self.card_ranking = {'J': 1, 'Q': 2, 'K': 3}

  def receive_outcome(self, obs: Dict[str, Any]):
    """
    Receives the game outcome of the previous game.

    For this simple heuristic bot, this method does nothing as it
    does not adapt its strategy based on past games.
    """
    pass

  def restart(self, player_id: int):
    """
    Starts a new game of Leduc Poker, assigning the player position.

    Args:
        player_id: The player ID (0 or 1) for the new game.
    """
    self.player_id = player_id

  def act(self, obs: Dict[str, Any]) -> str:
    """
    Outputs an action given an observation.

    Args:
        obs: A dictionary containing the game state from the player's perspective.

    Returns:
        A string representing the chosen action: 'FOLD', 'CALL', or 'RAISE'.
    """
    player_view = obs.get('player_view', {})
    public_state = obs.get('public_state', {})
    legal_actions = player_view.get('legal_actions', [])

    if not legal_actions:
      # If there are no legal actions, we can't do anything.
      # This can happen at the end of a hand.
      return 'CALL'

    hand = player_view.get('hand')
    round_name = public_state.get('round')

    if round_name == 'PREFLOP':
      return self._preflop_strategy(hand, legal_actions)
    elif round_name == 'POSTFLOP':
      public_card = public_state.get('public_card')
      return self._postflop_strategy(hand, public_card, legal_actions)

    # Default action if something unexpected happens
    return 'CALL' if 'CALL' in legal_actions else random.choice(legal_actions)

  def _preflop_strategy(self, hand: str, legal_actions: list[str]) -> str:
    """Determines the action for the pre-flop round."""
    hand_strength = self.card_ranking.get(hand, 0)

    # Strong hand (King) - play aggressively
    if hand_strength == 3:  # King
      if 'RAISE' in legal_actions:
        return 'RAISE'
      return 'CALL'

    # Medium hand (Queen) - play cautiously
    elif hand_strength == 2:  # Queen
      # If we can call, we do. Avoid folding if possible.
```


11 下注轮。它不会从对手行为中学习。12 """ 13 14 def__init__(self): 15 """初始化机器人。""" 16 self.player_id: int =-1 17 self.card_ranking = {'J': 1, 'Q': 2, 'K': 3} 18 19 def receive_outcome(self, obs: Dict[str, Any]): 20 """ 21 接收上一局的对局结果。22 23 对于这个简单的启发式机器人，此方法不执行任何操作，因为它 24 不会根据过去的博弈调整策略。25 """ 26 pass 27 28 def restart(self, player_id: int): 29 """ 30 开始一局新的 Leduc 扑克，并分配玩家位置。31 32 参数：33 player_id: 新一局的玩家标识符（0 或 1）。34 """ 35 self.player_id =player_id 36 37 def act(self, obs: Dict[str, Any]) ->str: 38 """ 39 根据给定的观测输出一个动作。40 41 参数：42 obs: 包含玩家视角下游戏状态的字典。43 44 回报：45 表示所选动作的字符串：'弃牌'、'跟注'或'加注'。46 """ 47 player_view = obs.get('player_view',{}) 48 public_state =obs.get('public_state', {}) 49 legal_actions =player_view.get('legal_actions', []) 50 51 if not legal_actions: 52 # 如果没有合法动作，就无法执行任何操作。53 # 一手牌结束时可能出现这种情况。54 return '跟注' 55 56 hand =player_view.get('hand') 57 round_name =public_state.get('round') 58 59 if round_name =='翻牌前':_60 return self._preflop_strategy(hand, legal_actions) 61 elif round_name =='翻牌后':_62 public_card =public_state.get('public_card') 63 return self._postflop_strategy(hand, public_card, legal_actions) 64 65 # 出现意外情况时的默认动作 66 return '跟注' if '跟注' in legal_actions else random.choice(legal_actions) 67 68 def _preflop_strategy(self, hand: str,legal_actions: list[str]) -> str: 69 """确定翻牌前轮的动作。""" 70 hand_strength =self.card_ranking.get(hand, 0) 71 72 # 强手牌（国王）——采用激进打法 73 if hand_strength == 3: # 国王 74 if '加注' in legal_actions: 75 return '加注' 76 return '跟注' 77 78 # 中等手牌（王后）——采用谨慎打法 79 elif hand_strength == 2: # 王后 80 # 如果可以跟注，就跟注。尽可能避免弃牌。

```
    if 'CALL' in legal_actions:
      return 'CALL'
    # This case should ideally not be hit if CALL is always an option
    # when FOLD is, but as a fallback.
    return 'FOLD'

  # Weak hand (Jack) - play defensively
  else: # Jack
    # We want to see the flop cheaply.
    if 'CALL' in legal_actions:
      return 'CALL'
    return 'FOLD'

def _postflop_strategy(self, hand: str, public_card: str, legal_actions: list[str]) -> str:
  """Determines the action for the post-flop round."""
  hand_strength = self.card_ranking.get(hand, 0)
  public_card_strength = self.card_ranking.get(public_card, 0)

  # Check for a pair (strongest hand)
  if hand == public_card:
    if 'RAISE' in legal_actions:
      return 'RAISE'
    return 'CALL'

  # High card strategy
  # If our private card is better than the public card, we have a decent high card hand
  if hand_strength > public_card_strength:
    if 'RAISE' in legal_actions:
      return 'RAISE'
    return 'CALL'

  # If our private card is weaker than the public card, our hand is weak.
  else:
    # Check-call if possible (zero-cost call)
    # A zero-cost call is implied if we just need to call 0 chips.
    # We will simplify and just check/call.
    if 'CALL' in legal_actions:
      return 'CALL'
    return 'FOLD'
```

81 if ’跟注’ in legal_actions: 82 return ’跟注’ 83 # 理想情况下，如果可弃牌时总是可以跟注，就不应触发这种情况，84 # 但这里将其作为后备方案。85 return ’弃牌’ 86 87 # 弱牌（杰克）——采取防守打法 88 else: # 杰克 89 # 我们希望以较低的成本看到翻牌。90 if ’跟注’ in legal_actions: 91 return ’跟注’ 92 return ’弃牌’ 93 94 def_postflop_strategy(self, hand: str, public_card: str, legal_actions: list[str]) ->str: 95 """确定翻牌后轮次的动作。""" 96 hand_strength =self.card_ranking.get(hand, 0) 97 public_card_strength = self.card_ranking.get(public_card, 0) 98 99 # 检查是否构成对子（最强牌型）100 if hand ==public_card: 101 if ’加注’ in legal_actions: 102 return ’加注’ 103 return ’跟注’ 104 105 # 高牌策略 106 # 如果我们的私有牌比公共牌大，就拥有不错的高牌牌型 107 if hand_strength >public_card_strength: 108 if ’加注’ in legal_actions: 109 return ’ 加注 ’ 110 return ’ 跟注 ’ 111 112 # 如果我们的私有牌比公共牌小，那么我们的手牌就较弱。113 else: 114 # 尽可能采用过牌跟注（零成本跟注）115 # 如果只需跟注 0 个筹码，就意味着这是一次零成本跟注。116 # 我们将简化处理，只进行过牌/跟注。117 if ’跟注’ in legal_actions: 118 return ’跟注’ 119 return ’弃牌’